1 Система непересекающихся множеств (DSU)

Поддерживает объединение компонент и проверку принадлежности одной компоненте; объединение возвращает признак изменения разбиения. Вход — векторы `parent` и `size` размера n, исходно $parent[i] = i$, $size[i] = 1$, вершины $0 \dots n-1$; требуется $0 \le n <$ `INT_MAX` и корректные индексы вершин. Поиск корня и объединение занимают амортизированно $O(\alpha(n))$ времени, память $O(n)$.

```
int find_root(vector<int>& parent, int vertex) {
    if (parent[vertex] == vertex) return vertex;
    return parent[vertex] = find_root(parent, parent[vertex]);
}

bool unite(vector<int>& parent, vector<int>& size, int first, int second) {
    first = find_root(parent, first);
    second = find_root(parent, second);
    if (first == second) return false;
    if (size[first] < size[second]) swap(first, second);
    parent[second] = first;
    size[first] += size[second];
    return true;
}
```

2 Дерево Фенвика

Поддерживает прибавление к позиции и суммы на полуинтервалах: префикс относится к $[0, right)$, запрос суммы — к $[left, right)$. Вход — изначально нулевой вектор `tree` размера $n+1$; требуется $0 \le n <$ `INT_MAX`, $0 \le left \le right \le n$, при обновлении $0 \le index < n$. Суммы и промежуточные значения должны помещаться в `ll`; каждая операция занимает $O(\log(n+1))$ времени и $O(1)$ дополнительной памяти.

```
void add(vector<ll>& tree, int index, ll delta) {
    int length = (int)tree.size() - 1;
    for (++index; index <= length;) {
        tree[index] += delta;
        int step = index & -index;
        if (step > length - index) break;
        index += step;
    }
}

ll prefix(const vector<ll>& tree, int right) {
    ll answer = 0;
    for (; right > 0; right -= right & -right) answer += tree[right];
    return answer;
}

ll range(const vector<ll>& tree, int left, int right) {
    return prefix(tree, right) - prefix(tree, left);
}
```

3 Дерево отрезков с отложенным прибавлением

Обрабатывает массив значений и запросы $(left, right, value, type)$: тип 0 прибавляет `value` на $[left, right)$, тип 1 возвращает сумму, игнорируя поле значения; ответ содержит суммы в порядке запросов. Требуется $0 \le n <$ `INT_MAX`$/4$, $0 \le left \le right \le n$; пустые массивы и полуинтервалы допустимы, суммы, метки и промежуточные произведения должны помещаться в `ll`. Построение занимает $O(n)$, запрос — $O(\log(n+1))$, общая память $O(n+q)$.

```
vector<ll> segmentTree(const vector<ll>& values,
        const vector<tuple<int, int, ll, int>>& queries) {
    int n = values.size();
    vector<ll> tree(4 * max(1, n)), lazy(tree.size()), answer;
    auto build = [&](auto&& self, int node, int left, int right) -> void {
        if (right - left == 1) { tree[node] = values[left]; return; }
        int middle = (left + right) / 2;
        self(self, node * 2, left, middle);
        self(self, node * 2 + 1, middle, right);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    };
    auto update = [&](auto&& self, int node, int left, int right,
                      int queryLeft, int queryRight, ll value) -> void {
        if (queryRight <= left || right <= queryLeft) return;
        if (queryLeft <= left && right <= queryRight) {
            tree[node] += value * (right - left); lazy[node] += value; return;
        }
        int middle = (left + right) / 2;
        self(self, node * 2, left, middle, queryLeft, queryRight, value);
        self(self, node * 2 + 1, middle, right, queryLeft, queryRight, value);
        tree[node] = tree[node * 2] + tree[node * 2 + 1]
                   + lazy[node] * (right - left);
    };
    auto query = [&](auto&& self, int node, int left, int right,
                     int queryLeft, int queryRight, ll carry) -> ll {
        if (queryRight <= left || right <= queryLeft) return 0;
        if (queryLeft <= left && right <= queryRight)
            return tree[node] + carry * (right - left);
```

```
        int middle = (left + right) / 2;
        carry += lazy[node];
        return self(self, node * 2, left, middle, queryLeft, queryRight, carry)
             + self(self, node * 2 + 1, middle, right, queryLeft, queryRight, carry);
    };
    if (n) build(build, 1, 0, n);
    for (auto [left, right, value, type] : queries)
        if (type == 0) { if (n) update(update, 1, 0, n, left, right, value); }
        else answer.push_back(n ? query(query, 1, 0, n, left, right, 0) : 0);
    return answer;
}
```

4 Система непересекающихся множеств с откатом

Изначально `parent[i]=i`, `size[i]=1`, история пуста; снимок — её размер. Объединение возвращает признак успеха, откат восстанавливает снимок только текущей ветви истории. Без сжатия путей поиск и объединение стоят $O(\log n)$, откат — $O(1)$ на отменённое объединение; память $O(n)$.

```
int rollback_root(const vector<int>& parent, int vertex) {
    while (parent[vertex] != vertex) vertex = parent[vertex];
    return vertex;
}
bool rollback_unite(vector<int>& parent, vector<int>& size,
        vector<array<int, 3>>& history, int first, int second) {
    first = rollback_root(parent, first);
    second = rollback_root(parent, second);
    if (first == second) return false;
    if (size[first] < size[second]) swap(first, second);
    history.push_back({second, first, size[first]});
    parent[second] = first;
    size[first] += size[second];
    return true;
}
void rollback(vector<int>& parent, vector<int>& size,
        vector<array<int, 3>>& history, int snapshot) {
    assert(0 <= snapshot && snapshot <= int(history.size()));
    while (int(history.size()) > snapshot) {
        auto [child, root, oldSize] = history.back();
        history.pop_back();
        parent[child] = child;
        size[root] = oldSize;
    }
}
```

5 Система непересекающихся множеств с чётностью

Ограничение (u, v, p) задаёт $color[u] \operatorname{xor} color[v] = p$, $p \in \{0, 1\}$; вершины нумеруются с нуля. Возвращает совместность каждого префикса ограничений; после противоречия все ответы ложны, для двудольности берут $p = 1$. Время $O((n + q)\alpha(n))$, память $O(n + q)$.

```
vector<bool> parity_dsu(int n, const vector<array<int, 3>>& edges) {
    vector<int> parent(n), size(n, 1), parity(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto&& self, int vertex) -> int {
        if (parent[vertex] == vertex) return vertex;
        int oldParent = parent[vertex];
        parent[vertex] = self(self, oldParent);
        parity[vertex] ^= parity[oldParent];
        return parent[vertex];
    };
    vector<bool> answer;
    bool valid = true;
    for (auto [first, second, required] : edges) {
        int a = find(find, first), b = find(find, second);
        int difference = parity[first] ^ parity[second] ^ required;
        if (a == b) valid = valid && difference == 0;
        else {
            if (size[a] < size[b]) swap(a, b);
            parent[b] = a;
            parity[b] = difference;
            size[a] += size[b];
        }
        answer.push_back(valid);
    }
    return answer;
}
```

6 Два дерева Фенвика: прибавление и сумма на отрезке

Массив изначально содержит n нулей; запрос $(type, left, right, delta)$ типа 0 прибавляет `delta` на $[left, right)$, типа 1 возвращает сумму этого отрезка. Ответы выдаются только для типа 1; требуется $0 \le n <$ `INT_MAX`, все суммы, отрицание `delta` и произведения на индексы помещаются в `ll`. Время $O((n + q)\log(n + 1))$, память $O(n + q)$.

```
vector<ll> fenwick_range(int n,
        const vector<tuple<int, int, int, ll>>& queries) {
```

```
    vector<ll> slope(n + 1), offset(n + 1), answer;
    auto add = [&](vector<ll>& tree, int index, ll delta) {
        for (++index; index <= n;) {
            tree[index] += delta;
            int step = index & -index;
            if (step > n - index) break;
            index += step;
        }
    };
    auto prefix = [&](int right) {
        ll a = 0, b = 0;
        for (int index = right; index > 0; index -= index & -index) {
            a += slope[index]; b += offset[index];
        }
        return a * right - b;
    };
    for (auto [type, left, right, delta] : queries) {
        if (type == 0) {
            add(slope, left, delta); add(slope, right, -delta);
            add(offset, left, delta * left);
            add(offset, right, -delta * right);
        } else answer.push_back(prefix(right) - prefix(left));
    }
    return answer;
}
```

7 Поиск порядковой статистики в дереве Фенвика

Для массива неотрицательных частот и рангов $k \geq 1$ возвращает индексы первых префиксов с суммой $\geq k$; если общей суммы не хватает, ответ n. Пустой массив допустим; $n <$ `INT_MAX`, частоты и их сумма помещаются в `ll`. Построение $O(n)$, запрос $O(\log(n+1))$, память $O(n+q)$.

```
vector<int> fenwick_kth(const vector<ll>& frequencies,
        const vector<ll>& ranks) {
    int n = frequencies.size();
    vector<ll> tree(n + 1);
    for (int i = 1; i <= n; ++i) {
        tree[i] += frequencies[i - 1];
        int step = i & -i;
        if (step <= n - i) tree[i + step] += tree[i];
    }
    int highest = 1;
    while (highest <= n / 2) highest *= 2;
    vector<int> answer;
    for (ll rank : ranks) {
        assert(rank >= 1);
        int index = 0;
        for (int step = highest; step > 0; step /= 2) {
            if (step <= n - index && tree[index + step] < rank) {
                index += step;
                rank -= tree[index];
            }
        }
        answer.push_back(index);
    }
    return answer;
}
```

8 Разреженная таблица минимумов

Для неизменяемого массива возвращает минимумы на непустых полуинтервалах $[left, right)$; пустой массив допустим только без запросов. Построение и память $O(n \log(n+1))$, запрос $O(1)$, ответы занимают $O(q)$ памяти.

```
vector<ll> sparse_table(const vector<ll>& values,
        const vector<pair<int, int>>& queries) {
    int n = values.size();
    vector<int> logarithm(n + 1);
    for (int i = 2; i <= n; ++i) logarithm[i] = logarithm[i / 2] + 1;
    vector<vector<ll>> table(logarithm[n] + 1, values);
    for (int level = 1; level <= logarithm[n]; ++level) {
        int length = 1 << level, half = length / 2;
        for (int i = 0; i <= n - length; ++i)
            table[level][i] = min(table[level - 1][i],
                                  table[level - 1][i + half]);
    }
    vector<ll> answer;
    for (auto [left, right] : queries) {
        int level = logarithm[right - left];
        answer.push_back(min(table[level][left],
                             table[level][right - (1 << level)]));
    }
    return answer;
}
```

9 Персистентное дерево: порядковая статистика отрезка

Для массива и запросов $(left, right, k)$ возвращает k-е по возрастанию значения на $[left, right)$ с учётом повторов. Требуется $1 \le k \le right - left$; при пустом массиве запросов нет, число узлов помещается в `int`. Время $O((n + q)\log(n+1))$, память $O(n\log(n+1) + q)$.

```
vector<ll> persistent_kth(const vector<ll>& values,
        const vector<array<int, 3>>& queries) {
    vector<ll> coordinates = values;
    sort(coordinates.begin(), coordinates.end());
    coordinates.erase(unique(coordinates.begin(), coordinates.end()),
                      coordinates.end());
    struct Node { int left = 0, right = 0, count = 0; };
    vector<Node> nodes(1);
    auto insert = [&](auto&& self, int old, int left, int right,
                      int position) -> int {
        int node = nodes.size();
        Node copy = nodes[old];
        nodes.push_back(copy);
        ++nodes[node].count;
        if (right - left > 1) {
            int middle = left + (right - left) / 2;
            if (position < middle) {
                int child = self(self, copy.left, left, middle, position);
                nodes[node].left = child;
            } else {
                int child = self(self, copy.right, middle, right, position);
                nodes[node].right = child;
            }
        }
        return node;
    };
    int length = coordinates.size();
    vector<int> roots(1, 0);
    for (ll value : values) {
        int position = lower_bound(coordinates.begin(), coordinates.end(), value)
                     - coordinates.begin();
        roots.push_back(insert(insert, roots.back(), 0, length, position));
    }
    vector<ll> answer;
    for (auto [left, right, rank] : queries) {
        int a = roots[left], b = roots[right], low = 0, high = length;
        while (high - low > 1) {
            int count = nodes[nodes[b].left].count - nodes[nodes[a].left].count;
            int middle = low + (high - low) / 2;
            if (rank <= count) {
                a = nodes[a].left; b = nodes[b].left; high = middle;
            } else {
                rank -= count;
                a = nodes[a].right; b = nodes[b].right; low = middle;
            }
        }
        answer.push_back(coordinates[low]);
    }
    return answer;
}
```

10 Динамическое дерево отрезков

Нулевой массив задан на целочисленном $[lower, upper)$; запрос $(type, left, right, delta)$ типа 0 прибавляет `delta` к позиции `left`, типа 1 возвращает сумму на $[left, right)$. Требуется $lower < upper$; разность границ и все суммы помещаются в `ll`, число узлов — в `int`. Время $O(q\log(upper - lower + 1))$, память $O(1 + u\log(upper - lower + 1) + q)$ для u обновлений; запросы узлов не создают.

```
vector<ll> dynamic_segment(ll lower, ll upper,
        const vector<tuple<int, ll, ll, ll>>& queries) {
    struct Node { int left = 0, right = 0; ll sum = 0; };
    vector<Node> nodes(2);
    auto add = [&](auto&& self, int node, ll left, ll right,
                   ll position, ll delta) -> void {
        nodes[node].sum += delta;
        if (right - left == 1) return;
        ll middle = left + (right - left) / 2;
        bool goLeft = position < middle;
        int child = goLeft ? nodes[node].left : nodes[node].right;
        if (!child) {
            child = nodes.size(); nodes.push_back(Node{});
            if (goLeft) nodes[node].left = child;
            else nodes[node].right = child;
        }
        self(self, child, goLeft ? left : middle, goLeft ? middle : right,
             position, delta);
    };
    auto sum = [&](auto&& self, int node, ll left, ll right,
```

```
                ll queryLeft, ll queryRight) -> ll {
        if (!node || queryRight <= left || right <= queryLeft) return 0;
        if (queryLeft <= left && right <= queryRight) return nodes[node].sum;
        ll middle = left + (right - left) / 2;
        return self(self, nodes[node].left, left, middle, queryLeft, queryRight)
             + self(self, nodes[node].right, middle, right, queryLeft, queryRight);
    };
    vector<ll> answer;
    for (auto [type, left, right, delta] : queries) {
        if (type == 0) add(add, 1, lower, upper, left, delta);
        else answer.push_back(sum(sum, 1, lower, upper, left, right));
    }
    return answer;
}
```

11 Монотонная очередь

Для массива возвращает минимумы всех окон длины $width$; требуется $1 \le width \le n$. Время $O(n)$, дополнительная память $O(width)$ без ответов.

```
vector<ll> monotonic_queue(const vector<ll>& values, int width) {
    int n = values.size();
    assert(1 <= width && width <= n);
    deque<int> queue;
    vector<ll> answer;
    for (int i = 0; i < n; ++i) {
        while (!queue.empty() && queue.front() <= i-width) queue.pop_front();
        while (!queue.empty() && values[queue.back()] >= values[i]) queue.pop_back();
        queue.push_back(i);
        if (i >= width-1) answer.push_back(values[queue.front()]);
    }
    return answer;
}
```

12 Тяжёло-лёгкая декомпозиция дерева

Даны непустое неориентированное дерево с корнем 0 и значения вершин; запрос $(type, u, v, value)$ типа 0 присваивает `value` вершине u, типа 1 возвращает сумму на пути $u \dots v$ включительно. Требуется $n <$ `INT_MAX`; разности значений и все суммы помещаются в `ll`, нужен стек глубины $O(n)$. Построение $O(n \log n)$, изменение $O(\log n)$, запрос пути $O(\log^2 n)$, память $O(n+q)$.

```
vector<ll> hld(const vector<vector<int>>& graph, vector<ll> values,
        const vector<tuple<int, int, int, ll>>& queries) {
    int n = graph.size();
    vector<int> parent(n, -1), depth(n), size(n, 1), heavy(n, -1), head(n), position(n);
    auto sizes = [&](auto&& self, int vertex) -> void {
        for (int child : graph[vertex]) if (child != parent[vertex]) {
            parent[child] = vertex; depth[child] = depth[vertex] + 1;
            self(self, child); size[vertex] += size[child];
            if (heavy[vertex] == -1 || size[child] > size[heavy[vertex]])
                heavy[vertex] = child;
        }
    };
    sizes(sizes, 0);
    int timer = 0;
    auto decompose = [&](auto&& self, int vertex, int chain) -> void {
        head[vertex] = chain; position[vertex] = timer++;
        if (heavy[vertex] != -1) self(self, heavy[vertex], chain);
        for (int child : graph[vertex])
            if (child != parent[vertex] && child != heavy[vertex]) self(self, child, child);
    };
    decompose(decompose, 0, 0);
    vector<ll> tree(n + 1), answer;
    auto add = [&](int index, ll delta) {
        for (++index; index <= n;) {
            tree[index] += delta;
            int step = index & -index;
            if (step > n-index) break;
            index += step;
        }
    };
    auto prefix = [&](int right) {
        ll sum = 0;
        for (; right; right -= right & -right) sum += tree[right];
        return sum;
    };
    for (int i = 0; i < n; ++i) add(position[i], values[i]);
    for (auto [type, u, v, value] : queries) {
        if (type == 0) {
            add(position[u], value-values[u]); values[u] = value;
        } else {
            ll sum = 0;
            while (head[u] != head[v]) {
                if (depth[head[u]] < depth[head[v]]) swap(u, v);
```

```
                sum += prefix(position[u]+1) - prefix(position[head[u]]);
                u = parent[head[u]];
            }
            if (depth[u] > depth[v]) swap(u, v);
            sum += prefix(position[v]+1) - prefix(position[u]);
            answer.push_back(sum);
        }
    }
    return answer;
}
```

13 Центроидная декомпозиция

Для неориентированного дерева возвращает родителей в дереве центроидов; у корня родитель -1, пустой вход даёт пустой вектор. Время $O(n \log n)$, память $O(n)$, высота результата $O(\log n)$; рекурсивным обходам нужен стек глубины $O(n)$.

```
vector<int> centroid_decomposition(const vector<vector<int>>& graph) {
    int n = graph.size();
    vector<int> size(n), parent(n, -1);
    vector<bool> removed(n);
    auto measure = [&](auto&& self, int vertex, int previous) -> void {
        size[vertex] = 1;
        for (int child : graph[vertex]) if (child != previous && !removed[child]) {
            self(self, child, vertex); size[vertex] += size[child];
        }
    };
    auto locate = [&](auto&& self, int vertex, int previous, int total) -> int {
        for (int child : graph[vertex])
            if (child != previous && !removed[child] && size[child] > total/2)
                return self(self, child, vertex, total);
        return vertex;
    };
    auto build = [&](auto&& self, int vertex, int previous) -> void {
        measure(measure, vertex, -1);
        int center = locate(locate, vertex, -1, size[vertex]);
        parent[center] = previous; removed[center] = true;
        for (int child : graph[center]) if (!removed[child]) self(self, child, center);
    };
    if (n) build(build, 0, -1);
    return parent;
}
```

14 Виртуальное дерево

Для дерева с корнем 0 и наборов важных вершин строит минимальные деревья, замкнутые по LCA. Каждое ребро содержит предка, потомка и длину пути между ними. Номера вершин исходные, рёбра направлены от предка к потомку, длина — число исходных рёбер; повторы удаляются, пустой и одноэлементный наборы дают пустой список рёбер. Предобработка $O(n \log n)$, набор размера k — $O(k \log(k+1) + k \log n)$, память $O(n \log n + \sum k)$; нужен стек глубины $O(n)$.

```
vector<vector<tuple<int, int, int>>> virtual_tree(
        const vector<vector<int>>& graph, const vector<vector<int>>& groups) {
    int n = graph.size(), levels = 1, timer = 0;
    while ((1LL << levels) <= n) ++levels;
    vector<vector<int>> up(levels, vector<int>(n));
    vector<int> tin(n), tout(n), depth(n);
    auto dfs = [&](auto&& self, int vertex, int parent) -> void {
        tin[vertex] = timer++;
        up[0][vertex] = parent;
        for (int level = 1; level < levels; ++level)
            up[level][vertex] = up[level-1][up[level-1][vertex]];
        for (int child : graph[vertex]) if (child != parent) {
            depth[child] = depth[vertex]+1; self(self, child, vertex);
        }
        tout[vertex] = timer;
    };
    dfs(dfs, 0, 0);
    auto ancestor = [&](int a, int b) { return tin[a] <= tin[b] && tin[b] < tout[a]; };
    auto lca = [&](int a, int b) {
        if (ancestor(a, b)) return a;
        for (int level = levels-1; level >= 0; --level)
            if (!ancestor(up[level][a], b)) a = up[level][a];
        return up[0][a];
    };
    auto byEntry = [&](int a, int b) { return tin[a] < tin[b]; };
    vector<vector<tuple<int, int, int>>> answer;
    for (auto vertices : groups) {
        sort(vertices.begin(), vertices.end(), byEntry);
        vertices.erase(unique(vertices.begin(), vertices.end()), vertices.end());
        int count = vertices.size();
        for (int i = 1; i < count; ++i) vertices.push_back(lca(vertices[i-1], vertices[i]));
        sort(vertices.begin(), vertices.end(), byEntry);
        vertices.erase(unique(vertices.begin(), vertices.end()), vertices.end());
```

```
        vector<int> stack;
        vector<tuple<int, int, int>> edges;
        for (int vertex : vertices) {
            while (!stack.empty() && !ancestor(stack.back(), vertex)) stack.pop_back();
            if (!stack.empty()) edges.emplace_back(stack.back(), vertex,
                                                   depth[vertex]-depth[stack.back()]);
            stack.push_back(vertex);
        }
        answer.push_back(move(edges));
    }
    return answer;
}
```

15 Эйлеров порядок и интервалы поддеревьев

Для неориентированного дерева с корнем 0 возвращает $(tin, tout, order)$: поддерево v занимает $[tin[v], tout[v])$, `order[tin[v]]=v`. Это порядок входов, не полный эйлеров обход для LCA; пустой вход допустим. Время и память $O(n)$, рекурсии нет.

```
tuple<vector<int>, vector<int>, vector<int>> euler_subtree(
        const vector<vector<int>>& graph) {
    int n = graph.size();
    vector<int> tin(n), tout(n), order;
    if (!n) return {tin, tout, order};
    vector<array<int, 3>> stack{{0, -1, 0}};
    while (!stack.empty()) {
        auto [vertex, parent, exit] = stack.back(); stack.pop_back();
        if (exit) { tout[vertex] = order.size(); continue; }
        tin[vertex] = order.size(); order.push_back(vertex);
        stack.push_back({vertex, parent, 1});
        for (auto it = graph[vertex].rbegin(); it != graph[vertex].rend(); ++it)
            if (*it != parent) stack.push_back({*it, vertex, 0});
    }
    return {tin, tout, order};
}
```

16 Алгоритм Дейкстры

Находит кратчайшие пути от источника в ориентированном или неориентированном графе с неотрицательными весами и возвращает расстояния и родителей; 2^{62} означает недостижимость, путь восстанавливается по `parent` от цели к источнику с разворотом. Вход — списки дуг (to, w), $1 \le n <$ `INT_MAX`, длины списков меньше `INT_MAX`, источник и концы дуг — корректные индексы $0 \dots n-1$; требуется $0 \le w < 2^{62}$ и все конечные расстояния меньше 2^{62}. Петли и кратные рёбра допустимы; для неориентированного графа нужны обе дуги. Время $O(n + m \log(m+2))$, память $O(n+m)$.

```
pair<vector<ll>, vector<int>> dijkstra(
        const vector<vector<pair<int, ll>>>& graph, int source) {
    const ll infinity = 1LL << 62;
    int n = graph.size();
    vector<ll> distance(n, infinity);
    vector<int> parent(n, -1);
    using State = pair<ll, int>;
    priority_queue<State, vector<State>, greater<State>> queue;
    distance[source] = 0;
    queue.push({0, source});
    while (!queue.empty()) {
        auto [current, vertex] = queue.top();
        queue.pop();
        if (current != distance[vertex]) continue;
        for (auto [next, weight] : graph[vertex]) {
            if (weight >= infinity - current) continue;
            ll candidate = current + weight;
            if (candidate >= distance[next]) continue;
            distance[next] = candidate;
            parent[next] = vertex;
            queue.push({candidate, next});
        }
    }
    return {distance, parent};
}
```

17 Компоненты сильной связности: алгоритм Косарайю

Разбивает ориентированный граф на компоненты взаимно достижимых вершин и возвращает число компонент и их номера для каждой вершины; номера $0 \dots count-1$ топологически упорядочены, межкомпонентная дуга ведёт от меньшего к большему. Вход — списки смежности с корректными индексами $0 \dots n-1$, число вершин и длины списков меньше `INT_MAX`; пустой граф, петли и кратные дуги допустимы. Обходы итеративные, время и память $O(n+m)$, включая обратный граф.

```
pair<int, vector<int>> kosaraju(const vector<vector<int>>& graph) {
    int n = graph.size();
    vector<vector<int>> reverseGraph(n);
    for (int vertex = 0; vertex < n; ++vertex)
        for (int next : graph[vertex])
```

```
            reverseGraph[next].push_back(vertex);
    vector<char> visited(n);
    vector<int> order, component(n, -1);
    for (int root = 0; root < n; ++root) {
        if (visited[root]) continue;
        vector<pair<int, int>> stack = {{root, 0}};
        visited[root] = true;
        while (!stack.empty()) {
            int vertex = stack.back().first;
            int& index = stack.back().second;
            if (index == (int)graph[vertex].size()) {
                order.push_back(vertex);
                stack.pop_back();
            } else {
                int next = graph[vertex][index++];
                if (visited[next]) continue;
                visited[next] = true;
                stack.push_back({next, 0});
            }
        }
    }
    int count = 0;
    reverse(order.begin(), order.end());
    for (int root : order) {
        if (component[root] != -1) continue;
        vector<int> stack = {root};
        component[root] = count;
        while (!stack.empty()) {
            int vertex = stack.back();
            stack.pop_back();
            for (int next : reverseGraph[vertex]) {
                if (component[next] != -1) continue;
                component[next] = count;
                stack.push_back(next);
            }
        }
        ++count;
    }
    return {count, component};
}
```

18 Мосты: поиск в глубину с номерами рёбер

Находит рёбра неориентированного графа, удаление которых увеличивает число компонент связности; вход `edges` — пары концов, результат — номера мостов по позициям в `edges` без сортировки. Требуется $0 \le n, m <$ `INT_MAX` и корректные индексы концов $0 \dots n-1$; несвязность, пустой граф, петли и кратные рёбра допустимы. Время и память $O(n+m)$; глубина рекурсии до n, поэтому для глубоких графов нужен достаточный стек или итеративный поиск в глубину.

```
vector<int> bridges(int n, const vector<pair<int, int>>& edges) {
    vector<vector<pair<int, int>>> graph(n);
    for (int id = 0; id < (int)edges.size(); ++id) {
        auto [first, second] = edges[id];
        graph[first].push_back({second, id});
        graph[second].push_back({first, id});
    }
    vector<int> entry(n, -1), low(n), answer;
    int timer = 0;
    auto dfs = [&](auto&& self, int vertex, int parentEdge) -> void {
        entry[vertex] = low[vertex] = timer++;
        for (auto [next, id] : graph[vertex]) {
            if (id == parentEdge) continue;
            if (entry[next] != -1) {
                low[vertex] = min(low[vertex], entry[next]);
            } else {
                self(self, next, id);
                low[vertex] = min(low[vertex], low[next]);
                if (low[next] > entry[vertex]) answer.push_back(id);
            }
        }
    };
    for (int vertex = 0; vertex < n; ++vertex)
        if (entry[vertex] == -1) dfs(dfs, vertex, -1);
    return answer;
}
```

19 Наименьший общий предок: двоичные подъёмы

Находит наименьших общих предков для пар вершин из `queries` и возвращает ответы в порядке запросов; пустой список запросов допустим. Вход — связное неориентированное дерево без петель и кратных рёбер, $1 \le n <$ `INT_MAX`, корень, концы рёбер и вершины запросов — корректные индексы $0 \dots n-1$; индексы и глубины хранятся в `int`. Каждый вызов заново строит таблицу за $O(n \log n)$ без рекурсии, запрос занимает $O(\log n)$, память с ответами — $O(n \log n + q)$.

```
vector<int> lca(const vector<vector<int>>& graph,
```

```
        const vector<pair<int, int>>& queries, int root = 0) {
    int n = graph.size(), levels = 1;
    while ((1LL << levels) <= n) ++levels;
    vector<int> depth(n, -1), order = {root};
    vector<vector<int>> up(levels, vector<int>(n, root));
    depth[root] = 0;
    for (int index = 0; index < (int)order.size(); ++index) {
        int vertex = order[index];
        for (int next : graph[vertex]) {
            if (depth[next] != -1) continue;
            depth[next] = depth[vertex] + 1;
            up[0][next] = vertex;
            order.push_back(next);
        }
    }
    for (int level = 1; level < levels; ++level)
        for (int vertex = 0; vertex < n; ++vertex)
            up[level][vertex] = up[level - 1][up[level - 1][vertex]];
    vector<int> answers;
    answers.reserve(queries.size());
    for (auto [first, second] : queries) {
        if (depth[first] < depth[second]) swap(first, second);
        int difference = depth[first] - depth[second];
        for (int level = 0; level < levels; ++level)
            if ((difference >> level) & 1) first = up[level][first];
        if (first != second) {
            for (int level = levels - 1; level >= 0; --level) {
                if (up[level][first] == up[level][second]) continue;
                first = up[level][first];
                second = up[level][second];
            }
            first = up[0][first];
        }
        answers.push_back(first);
    }
    return answers;
}
```

20 Максимальный поток: алгоритм Диница

Вычисляет величину максимального потока в ориентированной сети по дугам $(from, to, capacity)$; исходные рёбра не меняются, каждый вызов считает поток с нуля. Требуется $2 \le n <$ `INT_MAX`, $2m <$ `INT_MAX` для числа исходных дуг m, корректные индексы $0 \dots n-1$, различные исток и сток, целые неотрицательные ёмкости с суммой не более 2^{62}; петли и кратные дуги допустимы. Время $O(n^2 m)$, память $O(n+m)$; глубина рекурсии до n, для длинных путей необходим достаточный стек.

```
ll dinic(int n, const vector<tuple<int, int, ll>>& edges,
         int source, int sink) {
    struct Edge { int to; ll capacity; };
    vector<Edge> residual;
    vector<vector<int>> graph(n);
    vector<int> level(n), pointer(n);
    for (auto [from, to, capacity] : edges) {
        int id = residual.size();
        residual.push_back({to, capacity});
        residual.push_back({from, 0});
        graph[from].push_back(id);
        graph[to].push_back(id ^ 1);
    }
    auto bfs = [&]() {
        fill(level.begin(), level.end(), -1);
        queue<int> queue;
        level[source] = 0;
        queue.push(source);
        while (!queue.empty()) {
            int vertex = queue.front();
            queue.pop();
            for (int id : graph[vertex]) {
                const auto& edge = residual[id];
                if (!edge.capacity || level[edge.to] != -1) continue;
                level[edge.to] = level[vertex] + 1;
                queue.push(edge.to);
            }
        }
        return level[sink] != -1;
    };
    auto dfs = [&](auto&& self, int vertex, ll pushed) -> ll {
        if (vertex == sink) return pushed;
        for (int& index = pointer[vertex];
             index < (int)graph[vertex].size(); ++index) {
            int id = graph[vertex][index];
            auto& edge = residual[id];
            if (!edge.capacity || level[edge.to] != level[vertex] + 1)
```

```
                continue;
            ll sent = self(self, edge.to, min(pushed, edge.capacity));
            if (!sent) continue;
            edge.capacity -= sent;
            residual[id ^ 1].capacity += sent;
            return sent;
        }
        return 0;
    };
    ll total = 0;
    while (bfs()) {
        fill(pointer.begin(), pointer.end(), 0);
        while (ll sent = dfs(dfs, source, 1LL << 62)) total += sent;
    }
    return total;
}
```

21 Топологическая сортировка: алгоритм Кана

Для списков смежности ориентированного графа возвращает топологический порядок всех вершин либо `nullopt` при цикле. Вершины нумеруются с нуля; пустой граф, петли и кратные рёбра допустимы, входящие степени меньше `INT_MAX`. Время и память $O(n+m)$.

```
optional<vector<int>> topological_kahn(const vector<vector<int>>& graph) {
    int n = graph.size();
    vector<int> indegree(n), order;
    for (const auto& adjacent : graph)
        for (int next : adjacent) ++indegree[next];
    for (int vertex = 0; vertex < n; ++vertex)
        if (indegree[vertex] == 0) order.push_back(vertex);
    for (int index = 0; index < (int)order.size(); ++index)
        for (int next : graph[order[index]])
            if (--indegree[next] == 0) order.push_back(next);
    if ((int)order.size() != n) return nullopt;
    return order;
}
```

22 Задача 2-SAT: восстановление значений переменных

Для n переменных и пар литералов `clauses` возвращает удовлетворяющие значения $0/1$ либо `nullopt` при противоречии. Литерал $2i$ означает x_i, $2i+1$ — $\neg x_i$; требуется $0 \le 2n <$ `INT_MAX`, литералы из $[0, 2n)$. Пустая формула допустима; время и память $O(n+m)$, где m — число дизъюнктов, рекурсии нет.

```
optional<vector<int>> two_sat(int n, const vector<pair<int, int>>& clauses) {
    int vertices = 2 * n;
    vector<vector<int>> graph(vertices), reversed(vertices);
    for (auto [first, second] : clauses) {
        graph[first ^ 1].push_back(second);
        reversed[second].push_back(first ^ 1);
        graph[second ^ 1].push_back(first);
        reversed[first].push_back(second ^ 1);
    }
    vector<char> visited(vertices);
    vector<int> order, component(vertices, -1);
    for (int root = 0; root < vertices; ++root) {
        if (visited[root]) continue;
        vector<pair<int, int>> stack = {{root, 0}};
        visited[root] = true;
        while (!stack.empty()) {
            int vertex = stack.back().first;
            int& index = stack.back().second;
            if (index == (int)graph[vertex].size()) {
                order.push_back(vertex);
                stack.pop_back();
            } else {
                int next = graph[vertex][index++];
                if (visited[next]) continue;
                visited[next] = true;
                stack.push_back({next, 0});
            }
        }
    }
    reverse(order.begin(), order.end());
    int count = 0;
    for (int root : order) {
        if (component[root] != -1) continue;
        vector<int> stack = {root};
        component[root] = count;
        while (!stack.empty()) {
            int vertex = stack.back();
            stack.pop_back();
            for (int next : reversed[vertex]) {
                if (component[next] != -1) continue;
                component[next] = count;
```

```
                stack.push_back(next);
            }
        }
        ++count;
    }
    vector<int> assignment(n);
    for (int variable = 0; variable < n; ++variable) {
        if (component[2 * variable] == component[2 * variable + 1])
            return nullopt;
        assignment[variable] =
            component[2 * variable] > component[2 * variable + 1];
    }
    return assignment;
}
```

23 Эйлеров путь: ориентированный и неориентированный граф

Для пар концов `edges` и флага `directed` возвращает вершины эйлерова пути и ID рёбер между ними либо `nullopt`, если пути нет; ID — позиция ребра во входе. Вершины нумеруются с нуля; петли, кратные рёбра и изолированные вершины допустимы, требуется $2m <$ `INT_MAX`. При $m = 0$ ответ содержит вершину 0 для $n > 0$, иначе оба массива пусты. Время и память $O(n + m)$, рекурсии нет.

```
optional<pair<vector<int>, vector<int>>> euler_trail(
        int n, const vector<pair<int, int>>& edges, bool directed) {
    int m = edges.size();
    vector<vector<pair<int, int>>> graph(n);
    vector<int> incoming(n), outgoing(n);
    for (int id = 0; id < m; ++id) {
        auto [from, to] = edges[id];
        graph[from].push_back({to, id});
        ++outgoing[from];
        ++incoming[to];
        if (!directed) {
            graph[to].push_back({from, id});
            ++outgoing[to];
            ++incoming[from];
        }
    }
    if (m == 0) return pair{n == 0 ? vector<int>{} : vector<int>{0},
                            vector<int>{}};
    int start = edges[0].first, positive = 0, negative = 0, odd = 0;
    for (int vertex = 0; vertex < n; ++vertex) {
        if (directed) {
            int difference = outgoing[vertex] - incoming[vertex];
            if (difference == 1) { ++positive; start = vertex; }
            else if (difference == -1) ++negative;
            else if (difference != 0) return nullopt;
        } else if (outgoing[vertex] % 2 != 0) {
            ++odd;
            start = vertex;
        }
    }
    if (directed && !((positive == 0 && negative == 0) ||
                      (positive == 1 && negative == 1))) return nullopt;
    if (!directed && odd != 0 && odd != 2) return nullopt;
    vector<int> index(n), vertices, edgeIds;
    vector<char> used(m);
    vector<pair<int, int>> stack = {{start, -1}};
    while (!stack.empty()) {
        int vertex = stack.back().first;
        int& position = index[vertex];
        while (position < (int)graph[vertex].size() &&
               used[graph[vertex][position].second]) ++position;
        if (position == (int)graph[vertex].size()) {
            vertices.push_back(vertex);
            if (stack.back().second != -1)
                edgeIds.push_back(stack.back().second);
            stack.pop_back();
        } else {
            auto [next, id] = graph[vertex][position++];
            used[id] = true;
            stack.push_back({next, id});
        }
    }
    if ((int)edgeIds.size() != m) return nullopt;
    reverse(vertices.begin(), vertices.end());
    reverse(edgeIds.begin(), edgeIds.end());
    return pair{vertices, edgeIds};
}
```

24 Кратчайшие пути с весами ноль и один

Для graph с парами (конец дуги, вес) и источника возвращает кратчайшие расстояния типа ll; 2^{62} означает недостижимость. Требуются $n \geq 1$, корректные номера вершин с нуля и веса только 0 или 1. Время и память $O(n+m)$.

```
vector<ll> zero_one_bfs(
        const vector<vector<pair<int, ll>>>& graph, int source) {
    const ll infinity = 1LL << 62;
    vector<ll> distance(graph.size(), infinity);
    deque<pair<ll, int>> queue;
    distance[source] = 0;
    queue.push_back({0, source});
    while (!queue.empty()) {
        auto [current, vertex] = queue.front();
        queue.pop_front();
        if (current != distance[vertex]) continue;
        for (auto [next, weight] : graph[vertex]) {
            ll candidate = current + weight;
            if (candidate >= distance[next]) continue;
            distance[next] = candidate;
            if (weight == 0) queue.push_front({candidate, next});
            else queue.push_back({candidate, next});
        }
    }
    return distance;
}
```

25 Беллман—Форд и достижимый отрицательный цикл

Для $n \geq 1$, дуг (начало, конец, вес) и источника возвращает distance, parentEdge, cycleEdges — ID рёбер достижимого отрицательного цикла в прямом порядке. Если цикл не найден, расстояния кратчайшие, 2^{62} означает недостижимость; иначе расстояния — промежуточные метки, вершины с расстоянием $-\infty$ не помечаются. Все вычисляемые суммы строго между -2^{62} и 2^{62}; достаточно математического условия $n \max(1,m) W < 2^{62}$ при $|w| \leq W$. Время $O(nm+n)$, дополнительная память $O(n)$.

```
auto bellman_ford(int n, const vector<tuple<int, int, ll>>& edges,
        int source) {
    struct Result {
        vector<ll> distance;
        vector<int> parentEdge, cycleEdges;
    };
    const ll infinity = 1LL << 62;
    Result result{vector<ll>(n, infinity), vector<int>(n, -1), {}};
    result.distance[source] = 0;
    int changed = -1;
    for (int pass = 0; pass < n; ++pass) {
        changed = -1;
        for (int id = 0; id < (int)edges.size(); ++id) {
            auto [from, to, weight] = edges[id];
            if (result.distance[from] == infinity) continue;
            ll candidate = result.distance[from] + weight;
            if (candidate >= result.distance[to]) continue;
            result.distance[to] = candidate;
            result.parentEdge[to] = id;
            changed = to;
        }
        if (changed == -1) break;
    }
    if (changed == -1) return result;
    int vertex = changed;
    for (int step = 0; step < n; ++step)
        vertex = get<0>(edges[result.parentEdge[vertex]]);
    int start = vertex;
    do {
        int id = result.parentEdge[vertex];
        result.cycleEdges.push_back(id);
        vertex = get<0>(edges[id]);
    } while (vertex != start);
    reverse(result.cycleEdges.begin(), result.cycleEdges.end());
    return result;
}
```

26 Минимальный остовный лес: алгоритм Краскала

Для неориентированных рёбер (конец, конец, вес) возвращает суммарный вес минимального остовного леса и ID выбранных рёбер. Вершины нумеруются с нуля; отрицательные веса, петли и кратные рёбра допустимы, пустой граф даёт нулевой вес и пустой список. Все промежуточные суммы весов помещаются в ll; время $O(n + m\log(m+1) + m\alpha(n))$, память $O(n+m)$.

```
pair<ll, vector<int>> kruskal(
        int n, const vector<tuple<int, int, ll>>& edges) {
    vector<int> parent(n), size(n, 1), order(edges.size()), chosen;
    iota(parent.begin(), parent.end(), 0);
    iota(order.begin(), order.end(), 0);
```

```
    auto root = [&](int vertex) {
        while (parent[vertex] != vertex) {
            parent[vertex] = parent[parent[vertex]];
            vertex = parent[vertex];
        }
        return vertex;
    };
    sort(order.begin(), order.end(), [&](int first, int second) {
        return pair{get<2>(edges[first]), first} <
               pair{get<2>(edges[second]), second};
    });
    ll weightSum = 0;
    for (int id : order) {
        auto [from, to, weight] = edges[id];
        int first = root(from), second = root(to);
        if (first == second) continue;
        if (size[first] < size[second]) swap(first, second);
        parent[second] = first;
        size[first] += size[second];
        chosen.push_back(id);
        weightSum += weight;
    }
    return {weightSum, chosen};
}
```

27 Алгоритм Куна и минимальное вершинное покрытие

Для списков соседей левых вершин `graph` и размера правой доли `rightCount` возвращает максимальное паросочетание `leftMatch`, `rightMatch` и минимальное вершинное покрытие `coverLeft`, `coverRight`. Доли нумеруются с нуля независимо, могут быть пустыми; кратные рёбра допустимы, партнёр -1 означает свободную вершину. Время $O(L(L+m)+R)$, дополнительная память $O(L+R)$, нужен стек глубины $O(L)$.

```
auto kuhn_cover(const vector<vector<int>>& graph, int rightCount) {
    struct Result {
        vector<int> leftMatch, rightMatch, coverLeft, coverRight;
    };
    int leftCount = graph.size();
    Result result{vector<int>(leftCount, -1),
                  vector<int>(rightCount, -1), {}, {}};
    vector<char> visited(leftCount);
    auto augment = [&](auto&& self, int left) -> bool {
        if (visited[left]) return false;
        visited[left] = true;
        for (int right : graph[left]) {
            int previous = result.rightMatch[right];
            if (previous == -1 || self(self, previous)) {
                result.leftMatch[left] = right;
                result.rightMatch[right] = left;
                return true;
            }
        }
        return false;
    };
    for (int left = 0; left < leftCount; ++left) {
        fill(visited.begin(), visited.end(), false);
        augment(augment, left);
    }
    vector<char> reachedLeft(leftCount), reachedRight(rightCount);
    vector<int> stack;
    for (int left = 0; left < leftCount; ++left) {
        if (result.leftMatch[left] != -1) continue;
        reachedLeft[left] = true;
        stack.push_back(left);
    }
    while (!stack.empty()) {
        int left = stack.back();
        stack.pop_back();
        for (int right : graph[left]) {
            if (right == result.leftMatch[left] || reachedRight[right])
                continue;
            reachedRight[right] = true;
            int next = result.rightMatch[right];
            if (next != -1 && !reachedLeft[next]) {
                reachedLeft[next] = true;
                stack.push_back(next);
            }
        }
    }
    for (int left = 0; left < leftCount; ++left)
        if (!reachedLeft[left]) result.coverLeft.push_back(left);
    for (int right = 0; right < rightCount; ++right)
        if (reachedRight[right]) result.coverRight.push_back(right);
    return result;
```

```
}
```

28 Паросочетание: Хопкрофт–Карп

`graph[u]` содержит номера соседей левой вершины среди `rightCount` правых вершин; возвращает массивы партнёров максимального паросочетания слева и справа, -1 означает свободную вершину. Доли нумеруются с нуля независимо; пустые доли и кратные рёбра допустимы, $n + 1 <$ `INT_MAX`. Время $O((V + E)\sqrt{V + 1})$, дополнительная память $O(V)$, нужен стек глубины $O(V)$.

```
pair<vector<int>, vector<int>> hopcroft_karp(
        const vector<vector<int>>& graph, int rightCount) {
    int n = graph.size();
    vector<int> left(n, -1), right(rightCount, -1), distance(n), position(n);
    while (true) {
        queue<int> queue;
        fill(distance.begin(), distance.end(), -1);
        fill(position.begin(), position.end(), 0);
        for (int u = 0; u < n; ++u)
            if (left[u] == -1) { distance[u] = 0; queue.push(u); }
        int shortest = n + 1;
        while (!queue.empty()) {
            int u = queue.front(); queue.pop();
            if (distance[u] >= shortest) continue;
            for (int v : graph[u]) {
                int next = right[v];
                if (next == -1) shortest = distance[u] + 1;
                else if (distance[next] == -1) {
                    distance[next] = distance[u] + 1;
                    queue.push(next);
                }
            }
        }
        if (shortest == n + 1) break;
        auto augment = [&](auto&& self, int u) -> bool {
            for (int& i = position[u]; i < (int)graph[u].size(); ++i) {
                int v = graph[u][i], next = right[v];
                if ((next == -1 && distance[u] + 1 == shortest) ||
                    (next != -1 && distance[next] == distance[u] + 1 &&
                     self(self, next))) {
                    left[u] = v; right[v] = u;
                    return true;
                }
            }
            distance[u] = -1;
            return false;
        };
        for (int u = 0; u < n; ++u)
            if (left[u] == -1) augment(augment, u);
    }
    return {left, right};
}
```

29 Прямоугольное назначение: венгерский алгоритм

Для прямоугольной матрицы стоимостей $n \times m$ возвращает минимальную стоимость назначения каждой строки разному столбцу и номера выбранных столбцов. При $n > m$ возвращает `nullopt`, пустой вход даёт нулевую стоимость; все назначения разрешены, отрицательные стоимости допустимы. Требуется $n + 1, m + 1 <$ `INT_MAX`; вся арифметика помещается в `ll`, конечные `slack` и `delta` меньше `LLONG_MAX`. Время $O(n^2 m)$, дополнительная память $O(n + m)$.

```
optional<pair<ll, vector<int>>> hungarian(const vector<vector<ll>>& cost) {
    int n = cost.size(), m = n == 0 ? 0 : cost[0].size();
    if (n > m) return nullopt;
    vector<ll> rowPotential(n + 1), columnPotential(m + 1);
    vector<int> matched(m + 1), previous(m + 1);
    for (int row = 1; row <= n; ++row) {
        matched[0] = row;
        int column = 0;
        vector<ll> slack(m + 1, LLONG_MAX);
        vector<char> used(m + 1);
        do {
            used[column] = true;
            int currentRow = matched[column], nextColumn = -1;
            ll delta = LLONG_MAX;
            for (int j = 1; j <= m; ++j) if (!used[j]) {
                ll reduced = cost[currentRow - 1][j - 1]
                           - rowPotential[currentRow] - columnPotential[j];
                if (reduced < slack[j]) {
                    slack[j] = reduced;
                    previous[j] = column;
                }
                if (slack[j] < delta) { delta = slack[j]; nextColumn = j; }
            }
            for (int j = 0; j <= m; ++j) {
                if (used[j]) {
```

```
                    rowPotential[matched[j]] += delta;
                    columnPotential[j] -= delta;
                } else if (slack[j] != LLONG_MAX) slack[j] -= delta;
            }
            if (nextColumn == -1)
                throw logic_error("no free column");
            column = nextColumn;
        } while (matched[column] != 0);
        do {
            int nextColumn = previous[column];
            matched[column] = matched[nextColumn];
            column = nextColumn;
        } while (column != 0);
    }
    vector<int> assignment(n);
    for (int j = 1; j <= m; ++j)
        if (matched[j] != 0) assignment[matched[j] - 1] = j - 1;
    ll total = 0;
    for (int i = 0; i < n; ++i) total += cost[i][assignment[i]];
    return pair{total, assignment};
}
```

30 Поток минимальной стоимости с потенциалами

Для дуг (`from,to,capacity,cost`) возвращает поток до `limit`, его минимальную стоимость и потоки по исходным ID; при недостижимом лимите поток максимальный. Требуются $s \neq t$, неотрицательные ёмкости и лимит, отсутствие любых отрицательных циклов положительной ёмкости в начальной остаточной сети; достижимый отрицательный цикл вызывает `domain_error`. Петли и кратные дуги допустимы; $2E <$ `INT_MAX`, стоимость не равна `LLONG_MIN`, вся арифметика помещается в `ll`, конечные расстояния меньше `LLONG_MAX`. Время $O(VE + K(E+V)\log(V+1))$ для K увеличений, память $O(V+E)$.

```
tuple<ll, ll, vector<ll>> min_cost_flow(int n,
        const vector<tuple<int, int, ll, ll>>& input, int s, int t, ll limit) {
    struct Edge { int from, to; ll capacity, cost; };
    vector<Edge> edges;
    vector<vector<int>> graph(n);
    for (auto [u, v, capacity, cost] : input) {
        int id = edges.size();
        graph[u].push_back(id); graph[v].push_back(id + 1);
        edges.push_back({u, v, capacity, cost});
        edges.push_back({v, u, 0, -cost});
    }
    vector<ll> potential(n, LLONG_MAX);
    potential[s] = 0;
    for (int iteration = 0; iteration < n; ++iteration) {
        bool changed = false;
        for (const auto& edge : edges) {
            if (edge.capacity == 0 || potential[edge.from] == LLONG_MAX) continue;
            ll candidate = potential[edge.from] + edge.cost;
            if (candidate < potential[edge.to]) {
                potential[edge.to] = candidate;
                changed = true;
            }
        }
        if (!changed) break;
        if (iteration == n - 1) throw domain_error("reachable negative cost cycle");
    }
    for (ll& value : potential) if (value == LLONG_MAX) value = 0;
    ll flow = 0, total = 0;
    while (flow < limit) {
        vector<ll> distance(n, LLONG_MAX);
        vector<int> previous(n, -1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> queue;
        distance[s] = 0; queue.push({0, s});
        while (!queue.empty()) {
            auto [current, u] = queue.top(); queue.pop();
            if (current != distance[u]) continue;
            for (int id : graph[u]) {
                const auto& edge = edges[id];
                if (edge.capacity == 0) continue;
                ll reduced = edge.cost + potential[u] - potential[edge.to];
                ll candidate = current + reduced;
                if (candidate < distance[edge.to]) {
                    distance[edge.to] = candidate;
                    previous[edge.to] = id;
                    queue.push({candidate, edge.to});
                }
            }
        }
        if (previous[t] == -1) break;
        for (int v = 0; v < n; ++v)
            if (distance[v] != LLONG_MAX) potential[v] += distance[v];
        ll increase = limit - flow;
```

```
        for (int v = t; v != s; v = edges[previous[v]].from)
            increase = min(increase, edges[previous[v]].capacity);
        for (int v = t; v != s; v = edges[previous[v]].from) {
            int id = previous[v];
            total += increase * edges[id].cost;
            edges[id].capacity -= increase;
            edges[id ^ 1].capacity += increase;
        }
        flow += increase;
    }
    vector<ll> edgeFlow(input.size());
    for (int id = 0; id < (int)input.size(); ++id)
        edgeFlow[id] = edges[2 * id + 1].capacity;
    return {flow, total, edgeFlow};
}
```

31 Допустимая циркуляция с нижними границами

Для дуг (from,to,lower,upper) возвращает потоки по ID с сохранением баланса каждой вершины либо nullopt; источника и стока нет. Требуется $0 \le lower \le upper$, неверные границы дают nullopt; петли и пустая сеть допустимы. Требуется $n + 2 <$ INT_MAX, $2(E + n) <$ INT_MAX; вся арифметика помещается в ll, баланс не равен LLONG_MIN. Время $O(V^2(E + V))$, память $O(V + E)$, нужен стек глубины $O(V)$.

```
optional<vector<ll>> feasible_circulation(int n,
        const vector<tuple<int, int, ll, ll>>& input) {
    struct Edge { int to; ll capacity; };
    int source = n, sink = n + 1;
    vector<Edge> edges;
    vector<vector<int>> graph(n + 2);
    auto add = [&](int u, int v, ll capacity) {
        int id = edges.size();
        graph[u].push_back(id); graph[v].push_back(id + 1);
        edges.push_back({v, capacity}); edges.push_back({u, 0});
    };
    vector<ll> balance(n);
    for (auto [u, v, lower, upper] : input) {
        if (lower < 0 || lower > upper) return nullopt;
        add(u, v, upper - lower);
        balance[u] -= lower; balance[v] += lower;
    }
    ll required = 0;
    for (int v = 0; v < n; ++v) {
        if (balance[v] > 0) { add(source, v, balance[v]); required += balance[v]; }
        if (balance[v] < 0) add(v, sink, -balance[v]);
    }
    vector<int> level(n + 2), position(n + 2);
    ll flow = 0;
    while (flow < required) {
        fill(level.begin(), level.end(), -1);
        queue<int> queue;
        queue.push(source); level[source] = 0;
        while (!queue.empty()) {
            int u = queue.front(); queue.pop();
            for (int id : graph[u]) {
                int v = edges[id].to;
                if (edges[id].capacity > 0 && level[v] == -1) {
                    level[v] = level[u] + 1; queue.push(v);
                }
            }
        }
        if (level[sink] == -1) return nullopt;
        fill(position.begin(), position.end(), 0);
        auto push = [&](auto&& self, int u, ll amount) -> ll {
            if (u == sink) return amount;
            for (int& i = position[u]; i < (int)graph[u].size(); ++i) {
                int id = graph[u][i], v = edges[id].to;
                if (edges[id].capacity == 0 || level[v] != level[u] + 1) continue;
                ll sent = self(self, v, min(amount, edges[id].capacity));
                if (sent == 0) continue;
                edges[id].capacity -= sent; edges[id ^ 1].capacity += sent;
                return sent;
            }
            return 0;
        };
        while (flow < required) {
            ll sent = push(push, source, required - flow);
            if (sent == 0) break;
            flow += sent;
        }
    }
    vector<ll> result(input.size());
    for (int id = 0; id < (int)input.size(); ++id)
        result[id] = get<2>(input[id]) + edges[2 * id + 1].capacity;
```

```
    return result;
}
```

32 Точки сочленения и рёберные блоки двусвязности

Для неориентированного мультиграфа, заданного парами концов рёбер, возвращает флаги точек сочленения, блоки вершинной двусвязности как списки ID рёбер и изолированные вершины. ID — позиция ребра во входе; каждый мост и каждая петля образуют отдельный блок, изолированные вершины пустых блоков не создают. Требуется $2E <$ INT_MAX; время и память $O(V + E)$, нужен стек глубины $O(V)$.

```
tuple<vector<char>, vector<vector<int>>, vector<int>> biconnected_blocks(
        int n, const vector<pair<int, int>>& edges) {
    vector<vector<pair<int, int>>> graph(n);
    vector<vector<int>> blocks;
    for (int id = 0; id < (int)edges.size(); ++id) {
        auto [u, v] = edges[id];
        if (u == v) blocks.push_back({id});
        graph[u].push_back({v, id}); graph[v].push_back({u, id});
    }
    vector<int> tin(n, -1), low(n), stack, isolated;
    vector<char> articulation(n);
    int timer = 0;
    auto dfs = [&](auto&& self, int u, int parentEdge) -> void {
        tin[u] = low[u] = timer++;
        int children = 0;
        for (auto [v, id] : graph[u]) {
            if (id == parentEdge || v == u) continue;
            if (tin[v] == -1) {
                ++children; stack.push_back(id);
                self(self, v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u]) {
                    if (parentEdge != -1) articulation[u] = true;
                    vector<int> block;
                    do { block.push_back(stack.back()); stack.pop_back(); }
                    while (block.back() != id);
                    blocks.push_back(move(block));
                }
            } else if (tin[v] < tin[u]) {
                stack.push_back(id); low[u] = min(low[u], tin[v]);
            }
        }
        if (parentEdge == -1) articulation[u] = children > 1;
    };
    for (int u = 0; u < n; ++u) if (tin[u] == -1) {
        if (graph[u].empty()) isolated.push_back(u);
        dfs(dfs, u, -1);
    }
    return {articulation, blocks, isolated};
}
```

33 Офлайн-динамическая связность

Операция (type,u,v): 0 добавляет неориентированное ребро, 1 удаляет одну копию, 2 спрашивает связность; возвращает ответы $0/1$ в порядке запросов. Петли допустимы, копии удаляются в порядке LIFO; удаление отсутствующего ребра и неизвестный тип вызывают invalid_argument. Требуется $4\max(1, Q) <$ INT_MAX, число записей интервалов и размер истории помещаются в int. Время $O(Q \log(Q+1) \log(V+1))$, память $O(V + Q \log(Q+1))$, глубина рекурсии $O(\log(Q+1))$.

```
vector<int> offline_dynamic_connectivity(int n,
        const vector<tuple<int, int, int>>& operations) {
    int q = operations.size();
    if (q == 0) return {};
    vector<vector<pair<int, int>>> tree(4 * q);
    auto add = [&](auto&& self, int node, int l, int r, int a, int b,
                   pair<int, int> edge) -> void {
        if (a >= r || b <= l) return;
        if (a <= l && r <= b) { tree[node].push_back(edge); return; }
        int middle = l + (r - l) / 2;
        self(self, node * 2, l, middle, a, b, edge);
        self(self, node * 2 + 1, middle, r, a, b, edge);
    };
    map<pair<int, int>, vector<int>> active;
    for (int time = 0; time < q; ++time) {
        auto [type, u, v] = operations[time];
        if (u > v) swap(u, v);
        pair<int, int> edge = {u, v};
        if (type == 0) active[edge].push_back(time);
        else if (type == 1) {
            auto it = active.find(edge);
            if (it == active.end() || it->second.empty())
                throw invalid_argument("removing inactive edge");
            int start = it->second.back(); it->second.pop_back();
            add(add, 1, 0, q, start, time, edge);
```

```
        } else if (type != 2) throw invalid_argument("unknown operation type");
    }
    for (const auto& [edge, starts] : active)
        for (int start : starts) add(add, 1, 0, q, start, q, edge);
    vector<int> parent(n), size(n, 1), answers;
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](int v) {
        while (parent[v] != v) v = parent[v];
        return v;
    };
    vector<pair<int, int>> history;
    auto solve = [&](auto&& self, int node, int l, int r) -> void {
        int snapshot = history.size();
        for (auto [u, v] : tree[node]) {
            u = find(u); v = find(v);
            if (u == v) continue;
            if (size[u] < size[v]) swap(u, v);
            history.push_back({v, size[u]});
            parent[v] = u; size[u] += size[v];
        }
        if (r - l == 1) {
            auto [type, u, v] = operations[l];
            if (type == 2) answers.push_back(find(u) == find(v));
        } else {
            int middle = l + (r - l) / 2;
            self(self, node * 2, l, middle);
            self(self, node * 2 + 1, middle, r);
        }
        while ((int)history.size() > snapshot) {
            auto [v, oldSize] = history.back(); history.pop_back();
            size[parent[v]] = oldSize; parent[v] = v;
        }
    };
    solve(solve, 1, 0, q);
    return answers;
}
```

34 Префикс-функция

Для поиска подстрок и анализа периодичности возвращает массив π: на позиции i — длина наибольшего собственного префикса строки $s[0 \dots i]$, совпадающего с её суффиксом; $\pi[0] = 0$. Вход — строка из любых байтов, включая пустую, длины $n <$ INT_MAX; индексы с нуля, символы Юникода отдельно не декодируются. Время $O(n)$, память $O(n)$.

```
vector<int> prefixFunction(const string& text) {
    int length = text.size();
    vector<int> prefix(length);
    for (int index = 1; index < length; ++index) {
        int matched = prefix[index - 1];
        while (matched > 0 && text[index] != text[matched])
            matched = prefix[matched - 1];
        if (text[index] == text[matched]) ++matched;
        prefix[index] = matched;
    }
    return prefix;
}
```

35 Z-функция

Для поиска подстрок, границ и периодов возвращает массив z: на позиции i — длина наибольшего общего префикса всей строки и суффикса с позиции i; принято $z[0] = 0$. Вход — строка из любых байтов, включая пустую, длины $n <$ INT_MAX; индексы с нуля, сравниваются байты без декодирования Юникода. Время $O(n)$, память $O(n)$.

```
vector<int> zFunction(const string& text) {
    int length = text.size();
    vector<int> z(length);
    int left = 0, right = 0;
    for (int index = 1; index < length; ++index) {
        if (index <= right)
            z[index] = min(right - index + 1, z[index - left]);
        while (index + z[index] < length &&
               text[z[index]] == text[index + z[index]])
            ++z[index];
        if (index + z[index] - 1 > right) {
            left = index;
            right = index + z[index] - 1;
        }
    }
    return z;
}
```

36 Алгоритм Манакера: обе чётности

Находит радиусы всех максимальных палиндромов строки и возвращает массивы $(odd, even)$ за $O(n)$ времени и $O(n)$ памяти. Нечётный радиус $odd[i]$ включает центр i и задаёт полуинтервал $[i-odd[i]+1, i+odd[i])$ длины $2odd[i]-1$; чётный

$even[i]$ имеет центр между $i-1$ и i и задаёт $[i-even[i], i+even[i])$ длины $2even[i]$. Вход — любые байты, $n <$ INT_MAX, индексы с нуля, сравнение без декодирования Юникода; пустая строка даёт два пустых массива.

```
pair<vector<int>, vector<int>> manacher(const string& text) {
    int length = text.size();
    vector<int> odd(length), even(length);
    int left = 0, right = -1;
    for (int center = 0; center < length; ++center) {
        int radius = 1;
        if (center <= right)
            radius = min(odd[left + (right - center)], right - center + 1);
        while (center - radius >= 0 && center + radius < length &&
               text[center - radius] == text[center + radius])
            ++radius;
        odd[center] = radius;
        if (center + radius - 1 > right) {
            left = center - radius + 1;
            right = center + radius - 1;
        }
    }
    left = 0;
    right = -1;
    for (int center = 0; center < length; ++center) {
        int radius = 0;
        if (center <= right)
            radius = min(even[left + (right - center) + 1],
                         right - center + 1);
        while (center - radius - 1 >= 0 && center + radius < length &&
               text[center - radius - 1] == text[center + radius])
            ++radius;
        even[center] = radius;
        if (center + radius - 1 > right) {
            left = center - radius;
            right = center + radius - 1;
        }
    }
    return {odd, even};
}
```

37 Ахо-Корасик: число вхождений каждого шаблона

Считает вхождения набора шаблонов в текст, включая перекрытия, и возвращает vector<ll> в порядке шаблонов; дубликаты получают одинаковые ответы, пустой шаблон — $n+1$, пустые текст и набор допустимы. Алфавит строго a–z, некорректный байт вызывает invalid_argument; для суммы длин шаблонов L, их числа k и длины текста n требуется $L+1, k, n <$ INT_MAX. Автомат строится заново для каждого текста; при числе вершин бора $V \le L+1$ время $O(L + 26V + n + k)$, дополнительная память $O(26V + k)$.

```
vector<ll> ahoCounts(const vector<string>& patterns, const string& text) {
    array<int, 26> empty;
    empty.fill(-1);
    vector<array<int, 26>> next(1, empty);
    vector<int> fail(1);
    vector<ll> count(1);
    vector<int> terminal;
    auto letterIndex = [](char letter) {
        if (letter < 'a' || letter > 'z')
            throw invalid_argument("Ожидается буква a-z");
        return letter - 'a';
    };
    for (const string& pattern : patterns) {
        int vertex = 0;
        for (char letter : pattern) {
            int symbol = letterIndex(letter);
            if (next[vertex][symbol] == -1) {
                int child = (int)next.size();
                next.push_back(empty);
                fail.push_back(0);
                count.push_back(0);
                next[vertex][symbol] = child;
            }
            vertex = next[vertex][symbol];
        }
        terminal.push_back(vertex);
    }
    vector<int> order = {0};
    for (int index = 0; index < (int)order.size(); ++index) {
        int vertex = order[index];
        for (int symbol = 0; symbol < 26; ++symbol) {
            int child = next[vertex][symbol];
            int fallback = vertex == 0 ? 0 : next[fail[vertex]][symbol];
            if (child == -1) {
                next[vertex][symbol] = fallback;
            } else {
```

```
                fail[child] = fallback;
                order.push_back(child);
            }
        }
    }
    int vertex = 0;
    for (char letter : text) {
        vertex = next[vertex][letterIndex(letter)];
        ++count[vertex];
    }
    for (int index = (int)order.size() - 1; index > 0; --index) {
        int current = order[index];
        count[fail[current]] += count[current];
    }
    vector<ll> counts;
    for (int endpoint : terminal)
        counts.push_back(endpoint == 0 ? (ll)text.size() + 1 :
                         count[endpoint]);
    return counts;
}
```

38 Суффиксный массив удвоением и массив LCP

Для строки байтов возвращает `suffixOrder[i]` — начало i-го непустого суффикса в порядке `unsigned char`, и `lcp[i]` — длину общего префикса суффиксов рангов i и $i+1$. Размеры ответов n и $\max(0, n-1)$; пустая строка и нулевые байты допустимы, $n \le (\texttt{INT_MAX} - 3)/2$. Время $O(n \log(n+1))$, память $O(n)$.

```
pair<vector<int>, vector<int>> suffix_array_lcp(const string& text) {
    int n = int(text.size()), size = n + 1;
    vector<int> symbols(size), order(size), rank(size);
    for (int i = 0; i < n; ++i)
        symbols[i] = int(static_cast<unsigned char>(text[i])) + 1;
    vector<int> count(max(size, 257), 0);
    for (int symbol : symbols) ++count[symbol];
    for (int i = 1; i < 257; ++i) count[i] += count[i - 1];
    for (int i = size - 1; i >= 0; --i)
        order[--count[symbols[i]]] = i;
    int classes = 1;
    for (int i = 1; i < size; ++i) {
        if (symbols[order[i]] != symbols[order[i - 1]]) ++classes;
        rank[order[i]] = classes - 1;
    }
    vector<int> shifted(size), nextRank(size);
    for (int length = 1; length < size;) {
        for (int i = 0; i < size; ++i) {
            shifted[i] = order[i] - length;
            if (shifted[i] < 0) shifted[i] += size;
        }
        fill(count.begin(), count.begin() + classes, 0);
        for (int index : shifted) ++count[rank[index]];
        for (int i = 1; i < classes; ++i) count[i] += count[i - 1];
        for (int i = size - 1; i >= 0; --i)
            order[--count[rank[shifted[i]]]] = shifted[i];
        int nextClasses = 1;
        nextRank[order[0]] = 0;
        for (int i = 1; i < size; ++i) {
            int current = order[i], previous = order[i - 1];
            if (rank[current] != rank[previous] ||
                rank[(current + length) % size] !=
                rank[(previous + length) % size]) ++nextClasses;
            nextRank[current] = nextClasses - 1;
        }
        rank.swap(nextRank);
        classes = nextClasses;
        if (length >= size - length) break;
        length *= 2;
    }
    order.erase(order.begin());
    vector<int> position(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i) position[order[i]] = i;
    int matched = 0;
    for (int start = 0; start < n; ++start) {
        int place = position[start];
        if (place == n - 1) {
            matched = 0;
            continue;
        }
        int other = order[place + 1];
        while (start + matched < n && other + matched < n &&
               text[start + matched] == text[other + matched]) ++matched;
        lcp[place] = matched;
        if (matched > 0) --matched;
    }
```

```
    return {order, lcp};
}
```

39 Суффиксный автомат: различные подстроки и вхождения

Для строки `text` и образцов `patterns` возвращает число различных непустых подстрок и числа вхождений образцов в исходном порядке. Строки состоят из произвольных байтов; перекрытия учитываются, пустой образец имеет $n+1$ вхождений, отсутствующий — 0. Длины не превосходят $(\texttt{INT_MAX}-3)/2$, счётчики помещаются в `ll`; время $O((n+m)\log(\sigma+1)+q)$, память $O(n+q)$ при суммарной длине образцов m, их числе q и $\sigma \le 256$.

```
pair<ll, vector<ll>> substring_counts(const string& text,
        const vector<string>& patterns) {
    struct State {
        int length = 0, link = -1;
        map<unsigned char, int> next;
        ll occurrences = 0;
    };
    int n = int(text.size()), last = 0;
    vector<State> states(1);
    states.reserve(2 * n + 1);
    for (unsigned char symbol : text) {
        int current = int(states.size());
        states.emplace_back();
        states[current].length = states[last].length + 1;
        states[current].occurrences = 1;
        int parent = last;
        while (parent != -1 && !states[parent].next.count(symbol)) {
            states[parent].next[symbol] = current;
            parent = states[parent].link;
        }
        if (parent == -1) states[current].link = 0;
        else {
            int target = states[parent].next.at(symbol);
            if (states[parent].length + 1 == states[target].length)
                states[current].link = target;
            else {
                int clone = int(states.size());
                State copy = states[target];
                copy.length = states[parent].length + 1;
                copy.occurrences = 0;
                states.push_back(move(copy));
                while (parent != -1) {
                    auto edge = states[parent].next.find(symbol);
                    if (edge == states[parent].next.end() ||
                        edge->second != target) break;
                    edge->second = clone;
                    parent = states[parent].link;
                }
                states[target].link = states[current].link = clone;
            }
        }
        last = current;
    }
    int size = int(states.size());
    vector<int> count(n + 1), order(size);
    for (const auto& state : states) ++count[state.length];
    for (int i = 1; i <= n; ++i) count[i] += count[i - 1];
    for (int i = size - 1; i >= 0; --i)
        order[--count[states[i].length]] = i;
    ll distinct = 0;
    for (int i = size - 1; i > 0; --i) {
        int vertex = order[i], parent = states[vertex].link;
        states[parent].occurrences += states[vertex].occurrences;
        distinct += states[vertex].length - states[parent].length;
    }
    vector<ll> answer;
    for (const string& pattern : patterns) {
        if (pattern.empty()) {
            answer.push_back(ll(n) + 1);
            continue;
        }
        int vertex = 0;
        for (unsigned char symbol : pattern) {
            auto edge = states[vertex].next.find(symbol);
            if (edge == states[vertex].next.end()) {
                vertex = -1;
                break;
            }
            vertex = edge->second;
        }
        answer.push_back(vertex == -1 ? 0 : states[vertex].occurrences);
    }
    return {distinct, answer};
```

```
}
```

40 Минимальный циклический сдвиг: алгоритм Бута

Для строки возвращает наименьший индекс начала лексикографически минимального циклического сдвига; байты сравниваются как `unsigned char`. Для пустой строки ответ 0; требуется $n \le (\texttt{INT_MAX} - 3)/2$. Время $O(n)$, дополнительная память $O(1)$.

```
int minimal_cyclic_shift(const string& text) {
    int n = int(text.size());
    if (n == 0) return 0;
    int first = 0, second = 1, matched = 0;
    while (first < n && second < n && matched < n) {
        unsigned char a = static_cast<unsigned char>(text[(first + matched) % n]);
        unsigned char b = static_cast<unsigned char>(text[(second + matched) % n]);
        if (a == b) ++matched;
        else {
            if (a > b) {
                first += matched + 1;
                if (first <= second) first = second + 1;
            } else {
                second += matched + 1;
                if (second <= first) second = first + 1;
            }
            matched = 0;
        }
    }
    return min(first, second);
}
```

41 Бинарное возведение в степень по модулю

Вычисляет $a^e \bmod m$ за $O(\log(e+1))$ времени и $O(1)$ памяти, возвращая число из $[0, m)$. Основание — любое значение `int`, $0 \le e \le$ `LLONG_MAX`, модуль типа `int`: $1 \le m \le 2 \cdot 10^9$; произведения вычисляются в `ll` и меньше $4 \cdot 10^{18}$. При $e = 0$ результат равен $1 \bmod m$, в том числе для $a = 0$.

```
int modularPower(int base, ll exponent, int modulus) {
    assert(exponent >= 0 && modulus > 0 && modulus <= 2000000000);
    base %= modulus;
    if (base < 0) base += modulus;
    int result = 1 % modulus;
    while (exponent > 0) {
        if (exponent & 1)
            result = (ll)result * base % modulus;
        base = (ll)base * base % modulus;
        exponent >>= 1;
    }
    return result;
}
```

42 Расширенный алгоритм Евклида и обратный элемент

Расширенный алгоритм Евклида возвращает тройку (g, x, y) с $g = \gcd(a, b)$ и $ax + by = g$ для входов `int`, $0 \le a, b \le$ `INT_MAX`; для $(0, 0)$ результат $(0, 1, 0)$. Обратный элемент для любого `int` по модулю $2 \le m \le$ `INT_MAX` возвращается как `optional<int>`: пустой при $\gcd(value, m) \ne 1$, иначе число из $[0, m)$. Коэффициенты и промежуточные произведения вычисляются в `ll`: модули коэффициентов не превосходят `INT_MAX`, произведение частного и коэффициента — $\texttt{INT_MAX}^2$, вычитание также безопасно. Время $O(\log(\max(a, b) + 2))$, память $O(1)$.

```
tuple<int, ll, ll> extendedGcd(int first, int second) {
    assert(first >= 0 && second >= 0);
    ll firstX = 1, firstY = 0, secondX = 0, secondY = 1;
    while (second != 0) {
        int quotient = first / second;
        int remainder = first % second;
        first = second;
        second = remainder;
        ll nextX = firstX - quotient * secondX;
        ll nextY = firstY - quotient * secondY;
        firstX = secondX;
        firstY = secondY;
        secondX = nextX;
        secondY = nextY;
    }
    return {first, firstX, firstY};
}

optional<int> modularInverse(int value, int modulus) {
    assert(modulus >= 2);
    value %= modulus;
    if (value < 0) value += modulus;
    auto [divisor, coefficient, unused] = extendedGcd(value, modulus);
    if (divisor != 1) return nullopt;
    coefficient %= modulus;
    if (coefficient < 0) coefficient += modulus;
    return coefficient;
```

```
}
```

43 Линейное решето с минимальным простым делителем

Строит массив минимальных простых делителей чисел от 2 до входного n; для 0 и 1 записан ноль, при $x \geq 2$ число простое тогда и только тогда, когда `smallest[x] == x`. Требуется $0 \leq n <$ `INT_MAX` и достаточно памяти; время и память $O(n)$. Для факторизации $2 \leq x \leq n$ последовательно берут `smallest[x]` и делят x на него за $O(\log x)$.

```
vector<int> smallestPrimeFactors(int limit) {
    assert(limit >= 0 && limit < INT_MAX);
    vector<int> smallest(limit + 1), primes;
    for (int number = 2; number <= limit; ++number) {
        if (smallest[number] == 0) {
            smallest[number] = number;
            primes.push_back(number);
        }
        for (int prime : primes) {
            if (prime > smallest[number] || prime > limit / number) break;
            smallest[number * prime] = prime;
        }
    }
    return smallest;
}
```

44 Метод Гаусса для решения линейной системы

Решает вещественную систему $Ax = b$ по расширенной матрице из n строк по $m + 1$ чисел с b в последнем столбце; возвращает статус 0 (нет решения, вектор пуст), 1 (одно) или 2 (бесконечно много, возвращено одно со свободными переменными, равными нулю). Допустимы $n = 0$, $m = 0$; размеры меньше `INT_MAX`, все числа конечны, промежуточные вычисления не должны переполнять `db`. Порог `epsilon` конечный и положительный, сравнения ведущих элементов и невязки абсолютные; данные следует масштабировать, для плохо обусловленных систем статус и решение могут быть неточными. Время $O(nm \min(n, m) + nm)$, память $O(n(m + 1) + m)$ с копией входа.

```
pair<int, vector<db>> gauss(vector<vector<db>> matrix, int variables,
                            db epsilon = 1e-9) {
    assert(variables >= 0 && variables < INT_MAX);
    assert(isfinite(epsilon) && epsilon > 0 && matrix.size() < INT_MAX);
    int rows = matrix.size(), rank = 0;
    for (const auto& row : matrix) {
        assert(row.size() == static_cast<unsigned>(variables + 1));
        for (db value : row) assert(isfinite(value));
    }
    auto original = matrix;
    vector<int> pivotRow(variables, -1);
    for (int column = 0; column < variables && rank < rows; ++column) {
        int selected = rank;
        for (int row = rank + 1; row < rows; ++row)
            if (abs(matrix[row][column]) > abs(matrix[selected][column]))
                selected = row;
        if (abs(matrix[selected][column]) <= epsilon) continue;
        swap(matrix[selected], matrix[rank]);
        pivotRow[column] = rank;
        for (int entry = variables; entry >= column; --entry)
            matrix[rank][entry] /= matrix[rank][column];
        for (int row = 0; row < rows; ++row) {
            if (row == rank) continue;
            for (int entry = variables; entry >= column; --entry)
                matrix[row][entry] -= matrix[rank][entry] * matrix[row][column];
        }
        ++rank;
    }
    vector<db> solution(variables);
    for (int column = 0; column < variables; ++column)
        if (pivotRow[column] != -1)
            solution[column] = matrix[pivotRow[column]][variables];
    for (const auto& row : original) {
        db residual = -row[variables];
        for (int column = 0; column < variables; ++column)
            residual += row[column] * solution[column];
        if (abs(residual) > epsilon) return {0, {}};
    }
    return {rank == variables ? 1 : 2, solution};
}
```

45 Выпуклая оболочка целочисленных точек

Монотонная цепь строит минимальный выпуклый многоугольник, содержащий входные точки `pair<int, int>`; результат идёт против часовой стрелки от лексикографически минимальной точки без её повторения, дубликаты и промежуточные точки сторон удалены. Требуется $|x|, |y| \leq 10^9$ и число точек меньше `INT_MAX`; приведение к `ll` делается до вычитания, модули разностей не больше $2 \cdot 10^9$, произведений — $4 \cdot 10^{18}$, их разности — $8 \cdot 10^{18}$, что помещается в `ll`. Пустой вход даёт пустой ответ, одна уникальная точка возвращается сама, коллинеарный набор — два крайних конца в лексикографическом порядке. Время $O(n \log(n + 1))$, память $O(n)$.

```
vector<pair<int, int>> convexHull(vector<pair<int, int>> points) {
```

```
    assert(points.size() < INT_MAX);
    for (auto [x, y] : points) {
        assert(-1000000000 <= x && x <= 1000000000);
        assert(-1000000000 <= y && y <= 1000000000);
    }
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    if (points.size() <= 1) return points;
    auto cross = [](const auto& first, const auto& second, const auto& third) {
        ll dx = (ll)second.first - first.first;
        ll dy = (ll)second.second - first.second;
        return dx * ((ll)third.second - first.second)
             - dy * ((ll)third.first - first.first);
    };
    vector<pair<int, int>> lower, upper;
    for (const auto& point : points) {
        while (lower.size() >= 2 &&
               cross(lower[lower.size() - 2], lower.back(), point) <= 0)
            lower.pop_back();
        lower.push_back(point);
    }
    for (auto iterator = points.rbegin(); iterator != points.rend(); ++iterator) {
        while (upper.size() >= 2 &&
               cross(upper[upper.size() - 2], upper.back(), *iterator) <= 0)
            upper.pop_back();
        upper.push_back(*iterator);
    }
    lower.pop_back();
    upper.pop_back();
    lower.insert(lower.end(), upper.begin(), upper.end());
    return lower;
}
```

46 Безопасная модульная арифметика

Вычисляет сумму, произведение и степень по модулю без переполнения. Нужны $1 \le m \le$ LLONG_MAX, остатки $0 \le a, b < m$ и показатель $0 \le e \le$ LLONG_MAX. Умножение работает за $O(\log(b+1))$, степень — за $O(\log(e+1)\log m)$; память $O(1)$.

```
ll addMod(ll a, ll b, ll modulus) {
    assert(modulus > 0 && a >= 0 && a < modulus && b >= 0 && b < modulus);
    return a >= modulus - b ? a - (modulus - b) : a + b;
}
ll multiplyMod(ll a, ll b, ll modulus) {
    assert(modulus > 0 && a >= 0 && a < modulus && b >= 0 && b < modulus);
    ll result = 0;
    while (b) {
        if (b & 1) result = addMod(result, a, modulus);
        a = addMod(a, a, modulus);
        b >>= 1;
    }
    return result;
}
ll powerMod(ll a, ll exponent, ll modulus) {
    assert(modulus > 0 && a >= 0 && a < modulus && exponent >= 0);
    ll result = 1 % modulus;
    while (exponent) {
        if (exponent & 1) result = multiplyMod(result, a, modulus);
        a = multiplyMod(a, a, modulus);
        exponent >>= 1;
    }
    return result;
}
```

47 Китайская теорема об остатках без взаимной простоты

Объединяет сравнения $x \equiv a \pmod m$ и $x \equiv b \pmod n$, даже если модули не взаимно просты. Для остатков типа ll и $1 \le m, n \le$ INT_MAX возвращает минимальный неотрицательный остаток и НОК либо пустой optional, если решения нет. Время $O(\log \max(m, n))$; при последовательном объединении каждый входной модуль должен по-прежнему помещаться в int.

```
optional<pair<ll, ll>> chineseRemainder(ll a, int m, ll b, int n) {
    assert(m > 0 && n > 0);
    a %= m; if (a < 0) a += m;
    b %= n; if (b < 0) b += n;
    ll r = m, s = n, x = 1, y = 0;
    while (s) {
        ll q = r / s;
        ll nextR = r - q * s, nextX = x - q * y;
        r = s; s = nextR; x = y; y = nextX;
    }
    ll difference = b - a;
    if (difference % r) return nullopt;
    ll reduced = n / r, period = (ll)m * reduced;
```

```
    x %= reduced; if (x < 0) x += reduced;
    ll step = difference / r % reduced;
    if (step < 0) step += reduced;
    step = step * x % reduced;
    return pair<ll, ll>{(a + (ll)m * step) % period, period};
}
```

48 Miller–Rabin и Pollard rho

Проверяет простоту положительного `ll` и ищет делитель числа $n > 1$; для простого числа возвращает само число, а единица не считается простой. Нужны функции модульной арифметики `extra_math_mod` и внешний генератор случайных чисел. Проверка простоты детерминирована, а поиск делителя p требует ожидаемо $O(\sqrt{p})$ умножений по $O(\log n)$ каждое, без гарантии общего времени; память $O(1)$.

```
bool isPrime64(ll number) {
    assert(number > 0);
    if (number < 2) return false;
    for (int prime : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (number % prime == 0) return number == prime;
    }
    ll odd = number - 1;
    int shifts = 0;
    while (!(odd & 1)) { odd >>= 1; ++shifts; }
    for (ll base : {2LL, 325LL, 9375LL, 28178LL, 450775LL,
                    9780504LL, 1795265022LL}) {
        if (base % number == 0) continue;
        ll value = powerMod(base % number, odd, number);
        if (value == 1 || value == number - 1) continue;
        bool accepted = false;
        for (int i = 1; i < shifts; ++i) {
            value = multiplyMod(value, value, number);
            if (value == number - 1) { accepted = true; break; }
        }
        if (!accepted) return false;
    }
    return true;
}
ll pollardRho(ll number, mt19937_64& random) {
    assert(number > 1);
    if (number % 2 == 0) return 2;
    if (isPrime64(number)) return number;
    for (;;) {
        ll constant = uniform_int_distribution<ll>(1, number - 1)(random);
        ll slow = uniform_int_distribution<ll>(0, number - 1)(random);
        ll fast = slow, divisor = 1;
        auto advance = [&](ll value) {
            return addMod(multiplyMod(value, value, number), constant, number);
        };
        for (int iteration = 0; iteration < 200000 && divisor == 1; ++iteration) {
            slow = advance(slow);
            fast = advance(advance(fast));
            divisor = gcd(slow > fast ? slow - fast : fast - slow, number);
        }
        if (divisor > 1 && divisor < number) return divisor;
    }
}
```

49 Факторизация и перечисление делителей

Для $1 \le n \le$ `LLONG_MAX` строит разложение на отсортированные пары (p, e) и список всех положительных делителей по возрастанию. Нужны `extra_math_mod`, `extra_math_rho` и внешний генератор; для единицы разложение пусто, список делителей равен $\{1\}$. Время факторизации вероятностное, перечисление делителей занимает $O(\tau(n) \log \tau(n))$, память $O(\tau(n) + \log n)$.

```
vector<pair<ll, int>> factorize(ll number, mt19937_64& random) {
    assert(number > 0);
    vector<ll> pending{number}, primes;
    while (!pending.empty()) {
        ll value = pending.back(); pending.pop_back();
        if (value == 1) continue;
        if (isPrime64(value)) { primes.push_back(value); continue; }
        ll divisor = pollardRho(value, random);
        pending.push_back(divisor); pending.push_back(value / divisor);
    }
    sort(primes.begin(), primes.end());
    vector<pair<ll, int>> result;
    for (ll prime : primes) {
        if (result.empty() || result.back().first != prime)
            result.push_back({prime, 0});
        ++result.back().second;
    }
    return result;
}
vector<ll> divisors(ll number, mt19937_64& random) {
```

```
    auto factors = factorize(number, random);
    vector<ll> result{1};
    for (auto [prime, exponent] : factors) {
        int size = result.size();
        ll power = 1;
        for (int i = 1; i <= exponent; ++i) {
            power *= prime;
            for (int j = 0; j < size; ++j) result.push_back(result[j] * power);
        }
    }
    sort(result.begin(), result.end());
    return result;
}
```

50 Линейное решето Эйлера и Мёбиуса

По границе $0 \le n <$ INT_MAX строит таблицы функции Эйлера φ и функции Мёбиуса μ на $[0, n]$. При наличии индекса 1 оба значения равны 1, индекс 0 содержит технический ноль. Время и память $O(n)$; размер таблиц должен укладываться в лимит памяти.

```
pair<vector<int>, vector<int>> phiMobius(int limit) {
    assert(limit >= 0 && limit < INT_MAX);
    vector<int> phi(limit + 1), mu(limit + 1), smallest(limit + 1), primes;
    if (limit >= 1) phi[1] = mu[1] = 1;
    for (int value = 2; value <= limit; ++value) {
        if (!smallest[value]) {
            smallest[value] = value; primes.push_back(value);
            phi[value] = value - 1; mu[value] = -1;
        }
        for (int prime : primes) {
            if (prime > limit / value) break;
            int next = value * prime;
            smallest[next] = prime;
            if (value % prime == 0) {
                phi[next] = phi[value] * prime;
                break;
            }
            phi[next] = phi[value] * (prime - 1);
            mu[next] = -mu[value];
        }
    }
    return {phi, mu};
}
```

51 Факториалы и биномиальные коэффициенты по простому модулю

Предвычисляет факториалы и обратные факториалы до N за $O(N + \log p)$ и отвечает на запрос $C_n^k \bmod p$ за $O(1)$. Модуль $2 \le p \le 2 \cdot 10^9$ должен быть простым, а $0 \le n \le N < p$; простота не проверяется. Обе таблицы нужно брать из одного вызова с тем же модулем; для составного модуля или $N \ge p$ этот способ не подходит.

```
int primePower(int base, ll exponent, int prime) {
    assert(prime >= 2 && prime <= 2000000000 && exponent >= 0);
    base %= prime; if (base < 0) base += prime;
    int result = 1;
    while (exponent) {
        if (exponent & 1) result = (ll)result * base % prime;
        base = (ll)base * base % prime; exponent >>= 1;
    }
    return result;
}
pair<vector<int>, vector<int>> factorialTables(int limit, int prime) {
    assert(limit >= 0 && limit < prime && prime <= 2000000000);
    vector<int> factorial(limit + 1, 1), inverse(limit + 1, 1);
    for (int i = 1; i <= limit; ++i) factorial[i] = (ll)factorial[i - 1] * i % prime;
    inverse[limit] = primePower(factorial[limit], prime - 2, prime);
    for (int i = limit; i > 0; --i) inverse[i - 1] = (ll)inverse[i] * i % prime;
    return {factorial, inverse};
}
int binomial(int n, int k, int prime, const vector<int>& factorial,
             const vector<int>& inverse) {
    assert(prime >= 2 && prime <= 2000000000 && n >= 0 && n < prime);
    assert(factorial.size() == inverse.size() && n < (ll)factorial.size());
    if (k < 0 || k > n) return 0;
    return (ll)factorial[n] * inverse[k] % prime * inverse[n - k] % prime;
}
```

52 Блоки одинакового частного

Разбивает индексы $[1, n]$ на максимальные блоки (l, r, q) с одинаковым $\lfloor n/i \rfloor = q$ для суммирования сразу по блокам. Вход $0 \le n \le$ LLONG_MAX; для нуля возвращает пустой список. Время и память $O(\sqrt{n})$, поэтому число блоков должно быть практически допустимым.

```
vector<tuple<ll, ll, ll>> quotientBlocks(ll n) {
    assert(n >= 0);
    vector<tuple<ll, ll, ll>> result;
```

```
    for (ll left = 1; left <= n;) {
        ll quotient = n / left, right = n / quotient;
        result.emplace_back(left, right, quotient);
        if (right == n) break;
        left = right + 1;
    }
    return result;
}
```

53 NTT по модулю 998244353

Вычисляет прямое и обратное преобразования и свёртку полиномов по модулю 998244353, возвращая остатки коэффициентов. Коэффициенты должны лежать в $[0, 998244353)$, длина преобразования — степень двойки не больше 2^{23}, а для непустой свёртки $|a| + |b| - 1 \le 2^{23}$. Пустой множитель даёт пустой результат; время $O(N \log N)$, память $O(N)$.

```
void ntt(vector<int>& values, bool inverse) {
    int n = values.size(), modulus = 998244353;
    assert(n > 0 && n <= (1 << 23) && (n & (n - 1)) == 0);
    for (int value : values) assert(value >= 0 && value < modulus);
    auto power = [&](int base, int exponent) {
        int result = 1;
        while (exponent) {
            if (exponent & 1) result = (ll)result * base % modulus;
            base = (ll)base * base % modulus; exponent >>= 1;
        }
        return result;
    };
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(values[i], values[j]);
    }
    for (int length = 2; length <= n; length <<= 1) {
        int root = power(3, (modulus - 1) / length);
        if (inverse) root = power(root, modulus - 2);
        for (int start = 0; start < n; start += length) {
            int weight = 1;
            for (int j = 0; j < length / 2; ++j) {
                int a = values[start + j];
                int b = (ll)values[start + j + length / 2] * weight % modulus;
                values[start + j] = (a + b) % modulus;
                values[start + j + length / 2] = (a - b + modulus) % modulus;
                weight = (ll)weight * root % modulus;
            }
        }
    }
    if (inverse) {
        int scale = power(n, modulus - 2);
        for (int& value : values) value = (ll)value * scale % modulus;
    }
}
vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= (1 << 23));
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (ll)a[i] * b[i] % 998244353;
    ntt(a, true); a.resize(size);
    return a;
}
```

54 Гаусс над простым полем

Решает систему из n уравнений с m неизвестными по расширенной матрице из $m+1$ столбцов и возвращает статус и вектор решения. Статус 0 означает несовместность, 1 — единственное решение, 2 — несколько решений со свободными переменными, выбранными равными нулю. Нужны простой $2 \le p \le 10^9$, нормализованные остатки и `primePower` из `extra_math_binomial`; размеры меньше `INT_MAX`, нулевые допустимы, m задаётся явно. Время $O(nm \min(n, m) + \min(n, m) \log p + nm)$, память $O(n(m+1) + m)$.

```
pair<int, vector<int>> gaussPrime(vector<vector<int>> matrix, int variables,
                                  int prime) {
    assert(variables >= 0 && variables < INT_MAX && matrix.size() < INT_MAX);
    assert(prime >= 2 && prime <= 1000000000);
    for (const auto& row : matrix) {
        assert(row.size() == (ll)variables + 1);
        for (int value : row) assert(value >= 0 && value < prime);
    }
    int rows = matrix.size(), rank = 0;
    vector<int> pivot(variables, -1);
    for (int column = 0; column < variables && rank < rows; ++column) {
        int selected = rank;
```

```
        while (selected < rows && matrix[selected][column] == 0) ++selected;
        if (selected == rows) continue;
        swap(matrix[selected], matrix[rank]);
        int inverse = primePower(matrix[rank][column], prime - 2, prime);
        for (int j = column; j <= variables; ++j)
            matrix[rank][j] = (ll)matrix[rank][j] * inverse % prime;
        for (int i = 0; i < rows; ++i) if (i != rank) {
            int scale = matrix[i][column];
            for (int j = column; j <= variables; ++j)
                matrix[i][j] = (matrix[i][j] - (ll)scale * matrix[rank][j] % prime + prime) % prime;
        }
        pivot[column] = rank++;
    }
    for (int i = rank; i < rows; ++i) if (matrix[i][variables]) return {0, {}};
    vector<int> solution(variables);
    for (int j = 0; j < variables; ++j)
        if (pivot[j] != -1) solution[j] = matrix[pivot[j]][variables];
    return {rank == variables ? 1 : 2, solution};
}
```

55 Определитель над простым полем

Возвращает определитель квадратной матрицы по простому модулю $2 \le p \le 10^9$; определитель пустой матрицы равен 1. Коэффициенты должны быть нормализованы, $n <$ INT_MAX; нужна primePower из extra_math_binomial. Время $O(n^3 + n \log p)$, память $O(n^2)$ с копией матрицы; результат не является точным целочисленным определителем.

```
int determinant(vector<vector<int>> matrix, int prime) {
    assert(prime >= 2 && prime <= 1000000000 && matrix.size() < INT_MAX);
    int n = matrix.size(), result = 1;
    for (const auto& row : matrix) {
        assert(row.size() == matrix.size());
        for (int value : row) assert(value >= 0 && value < prime);
    }
    for (int column = 0; column < n; ++column) {
        int selected = column;
        while (selected < n && !matrix[selected][column]) ++selected;
        if (selected == n) return 0;
        if (selected != column) {
            swap(matrix[selected], matrix[column]); result = (prime - result) % prime;
        }
        result = (ll)result * matrix[column][column] % prime;
        int inverse = primePower(matrix[column][column], prime - 2, prime);
        for (int i = column + 1; i < n; ++i) {
            int scale = (ll)matrix[i][column] * inverse % prime;
            for (int j = column; j < n; ++j)
                matrix[i][j] = (matrix[i][j] - (ll)scale * matrix[column][j] % prime + prime) % prime;
        }
    }
    return result;
}
```

56 Умножение и степень матрицы

Перемножает квадратные матрицы одинакового размера и возводит матрицу в степень $0 \le e \le$ LLONG_MAX по модулю m. Нужны $1 \le m \le 2 \cdot 10^9$, коэффициенты в $[0, m)$ и $n <$ INT_MAX; нулевая степень даёт единичную матрицу по этому модулю, включая $m = 1$. Для степени время $O(n^3 \log(e+1) + n^2)$, память $O(n^2)$.

```
vector<vector<int>> matrixMultiply(const vector<vector<int>>& a,
                                   const vector<vector<int>>& b, int modulus) {
    assert(modulus >= 1 && modulus <= 2000000000 && a.size() < INT_MAX);
    assert(a.size() == b.size());
    int n = a.size();
    for (int i = 0; i < n; ++i) {
        assert(a[i].size() == a.size() && b[i].size() == a.size());
        for (int value : a[i]) assert(value >= 0 && value < modulus);
        for (int value : b[i]) assert(value >= 0 && value < modulus);
    }
    vector<vector<int>> result(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k)
            for (int j = 0; j < n; ++j)
                result[i][j] = (result[i][j] + (ll)a[i][k] * b[k][j]) % modulus;
    return result;
}
vector<vector<int>> matrixPower(vector<vector<int>> matrix, ll exponent, int modulus) {
    assert(exponent >= 0 && modulus >= 1 && modulus <= 2000000000);
    assert(matrix.size() < INT_MAX);
    int n = matrix.size();
    vector<vector<int>> result(n, vector<int>(n));
    for (int i = 0; i < n; ++i) {
        assert(matrix[i].size() == matrix.size());
        for (int value : matrix[i]) assert(value >= 0 && value < modulus);
        result[i][i] = 1 % modulus;
    }
```

```
    while (exponent) {
        if (exponent & 1) result = matrixMultiply(result, matrix, modulus);
        matrix = matrixMultiply(matrix, matrix, modulus); exponent >>= 1;
    }
    return result;
}
```

57 Целочисленные предикаты: cross, dot, ориентация

Вычисляет векторное и скалярное произведения и знак поворота для точек `pair<ll,ll>`; положительный поворот направлен против часовой стрелки, нулевой означает коллинеарность. Исходные координаты до вычитания должны удовлетворять $|x|, |y| \le 10^9$, иначе произведения могут переполнить `ll`. Нужны стандартные заголовки, `std` и `ll = long long`; время и память $O(1)$.

```
ll cross_integer(pair<ll, ll> a, pair<ll, ll> b) {
    return a.first * b.second - a.second * b.first;
}

ll dot_integer(pair<ll, ll> a, pair<ll, ll> b) {
    return a.first * b.first + a.second * b.second;
}

int orientation(pair<ll, ll> a, pair<ll, ll> b, pair<ll, ll> c) {
    ll value = cross_integer({b.first - a.first, b.second - a.second},
                             {c.first - a.first, c.second - a.second});
    return (value > 0) - (value < 0);
}
```

58 Точка на отрезке и пересечение отрезков

Проверяет принадлежность точки замкнутому отрезку и пересечение двух замкнутых отрезков, возвращая логический ответ. Касания концами и коллинеарное перекрытие считаются пересечением, вырожденный отрезок допустим. Нужна `orientation` с исходными целочисленными координатами по модулю не больше 10^9; время и память $O(1)$.

```
bool on_segment(pair<ll, ll> p, pair<ll, ll> a, pair<ll, ll> b) {
    return orientation(a, b, p) == 0 &&
        min(a.first, b.first) <= p.first && p.first <= max(a.first, b.first) &&
        min(a.second, b.second) <= p.second && p.second <= max(a.second, b.second);
}

bool segments_intersect(pair<ll, ll> a, pair<ll, ll> b,
                        pair<ll, ll> c, pair<ll, ll> d) {
    int ab_c = orientation(a, b, c), ab_d = orientation(a, b, d);
    int cd_a = orientation(c, d, a), cd_b = orientation(c, d, b);
    if (ab_c == 0 && on_segment(c, a, b)) return true;
    if (ab_d == 0 && on_segment(d, a, b)) return true;
    if (cd_a == 0 && on_segment(a, c, d)) return true;
    if (cd_b == 0 && on_segment(b, c, d)) return true;
    return ab_c * ab_d < 0 && cd_a * cd_b < 0;
}
```

59 Вещественные прямые: пересечение, проекция, отражение

Для точек `pair<db,db>` находит пересечение прямых, проекцию и отражение точки; прямая задаётся двумя различными точками. Пересечение возвращает `nullopt` для параллельных или совпадающих прямых, проекция и отражение — для вырожденной прямой. Здесь и далее `db = double`, координаты и промежуточные вычисления должны быть конечны, а `eps` — конечным и неотрицательным; допуск подбирается под задачу и не гарантирует точности на почти вырожденных данных. Время и память $O(1)$; параллельность проверяется относительным угловым допуском, совпадение концов — точно.

```
db cross_real(pair<db, db> a, pair<db, db> b, pair<db, db> c) {
    return (b.first - a.first) * (c.second - a.second) -
           (b.second - a.second) * (c.first - a.first);
}

db squared_distance(pair<db, db> a, pair<db, db> b) {
    db x = a.first - b.first, y = a.second - b.second;
    return x * x + y * y;
}

optional<pair<db, db>> line_intersection(
        pair<db, db> a, pair<db, db> b,
        pair<db, db> c, pair<db, db> d, db eps) {
    db ux = b.first - a.first, uy = b.second - a.second;
    db vx = d.first - c.first, vy = d.second - c.second;
    db determinant = ux * vy - uy * vx;
    if (a == b || c == d ||
        abs(determinant) <= eps * hypot(ux, uy) * hypot(vx, vy)) return nullopt;
    db t = ((c.first - a.first) * vy - (c.second - a.second) * vx) / determinant;
    return pair<db, db>{a.first + t * ux, a.second + t * uy};
}

optional<pair<db, db>> project_line(
        pair<db, db> p, pair<db, db> a, pair<db, db> b) {
```

```
    if (a == b) return nullopt;
    db x = b.first - a.first, y = b.second - a.second;
    db t = ((p.first - a.first) * x + (p.second - a.second) * y) / (x*x + y*y);
    return pair<db, db>{a.first + t*x, a.second + t*y};
}

optional<pair<db, db>> reflect_line(
        pair<db, db> p, pair<db, db> a, pair<db, db> b) {
    auto projection = project_line(p, a, b);
    if (!projection) return nullopt;
    return pair<db, db>{2*projection->first - p.first, 2*projection->second - p.second};
}
```

60 Расстояния от точки до прямой и отрезка

Возвращает евклидово расстояние от точки до прямой или замкнутого отрезка, заданных двумя точками. Для прямой из одной точки возвращает `nullopt`, для отрезка из одной точки — расстояние до неё. Нужны `cross_real` и `squared_distance` с конечной вещественной арифметикой; время и память $O(1)$.

```
optional<db> distance_line(pair<db, db> p, pair<db, db> a, pair<db, db> b) {
    if (a == b) return nullopt;
    return abs(cross_real(a, b, p)) / hypot(b.first-a.first, b.second-a.second);
}

db distance_segment(pair<db, db> p, pair<db, db> a, pair<db, db> b) {
    if (a == b) return sqrt(squared_distance(p, a));
    db x = b.first-a.first, y = b.second-a.second;
    db t = clamp(((p.first-a.first)*x + (p.second-a.second)*y) / (x*x+y*y), 0.0, 1.0);
    return hypot(p.first-a.first-t*x, p.second-a.second-t*y);
}
```

61 Ориентированная площадь многоугольника

По циклическому списку вершин возвращает ориентированную площадь: положительную при обходе против часовой стрелки, для обычной площади возьмите модуль. Пустой список, точка и отрезок дают ноль; для самопересечений результат алгебраический, а не площадь объединения. Нужна `cross_real` с конечной вещественной арифметикой; время $O(n)$, дополнительная память $O(1)$.

```
db polygon_area(const vector<pair<db, db>>& polygon) {
    db twice = 0;
    for (int i = 1; i + 1 < (int)polygon.size(); ++i)
        twice += cross_real(polygon[0], polygon[i], polygon[i+1]);
    return twice / 2;
}
```

62 Точка в произвольном простом многоугольнике

По целочисленным вершинам простого многоугольника и точке возвращает -1 снаружи, 0 на границе, 1 внутри за $O(n)$. Направление обхода произвольно, первую вершину в конце не повторяют; пустой список не содержит точек, одна или две точки задают только границу. Нужны `orientation` и `on_segment`, исходные координаты по модулю не больше 10^9.

```
int point_polygon(const vector<pair<ll, ll>>& polygon, pair<ll, ll> p) {
    bool inside = false;
    int n = polygon.size();
    for (int i = 0; i < n; ++i) {
        auto a = polygon[i], b = polygon[(i+1)%n];
        if (on_segment(p, a, b)) return 0;
        if ((a.second > p.second) != (b.second > p.second)) {
            int turn = orientation(a, b, p);
            if ((turn > 0) == (b.second > a.second)) inside = !inside;
        }
    }
    return inside ? 1 : -1;
}
```

63 Точка в выпуклом многоугольнике за логарифм

Проверяет точку в выпуклом многоугольнике за $O(\log n)$ и возвращает -1 снаружи, 0 на границе, 1 внутри. При $n \geq 3$ вершины должны идти против часовой стрелки без повторов и промежуточных коллинеарных точек; начальная вершина любая, пустой список, точка и отрезок допустимы. Нужны целочисленные предикаты с исходными координатами по модулю не больше 10^9; дополнительная память $O(1)$, выпуклость входа не проверяется.

```
int point_convex(const vector<pair<ll, ll>>& polygon, pair<ll, ll> p) {
    int n = polygon.size();
    if (n == 0) return -1;
    if (n == 1) return p == polygon[0] ? 0 : -1;
    if (n == 2) return on_segment(p, polygon[0], polygon[1]) ? 0 : -1;
    int first = orientation(polygon[0], polygon[1], p);
    int last = orientation(polygon[0], polygon[n-1], p);
    if (first < 0 || last > 0) return -1;
    if (first == 0) return on_segment(p, polygon[0], polygon[1]) ? 0 : -1;
    if (last == 0) return on_segment(p, polygon[0], polygon[n-1]) ? 0 : -1;
    int left = 1, right = n-1;
    while (right-left > 1) {
```

```
        int middle = (left+right)/2;
        if (orientation(polygon[0], polygon[middle], p) >= 0) left = middle;
        else right = middle;
    }
    return orientation(polygon[left], polygon[right], p);
}
```

64 Divide and conquer DP: монотонный argmin

По матрице стоимостей $c[l][r]$ размера $(n+1)^2$ и $0 \le k \le n$ вычисляет минимальные стоимости разбиения префиксов на ровно k непустых частей. Возвращает весь последний слой DP; недостижимые состояния равны LLONG_MAX/4, все конечные стоимости и суммы должны лежать строго между минус и плюс этой границей. Самый левый оптимальный разрез обязан не убывать с правой границей на каждом слое; это условие не проверяется. Время $O(kn \log n)$, дополнительная память $O(n)$ сверх входной матрицы.

```
vector<ll> divideConquerDP(const vector<vector<ll>>& cost, int k) {
    int n = (int)cost.size() - 1;
    assert(n >= 0 && k >= 0 && k <= n);
    for (const auto& row : cost) assert((int)row.size() == n + 1);
    ll inf = LLONG_MAX / 4;
    vector<ll> previous(n + 1, inf), current(n + 1, inf);
    previous[0] = 0;
    for (int layer = 1; layer <= k; ++layer) {
        fill(current.begin(), current.end(), inf);
        auto solve = [&](auto&& self, int left, int right,
                         int optLeft, int optRight) -> void {
            if (left > right) return;
            int middle = (left + right) / 2, best = optLeft;
            for (int split = optLeft; split <= min(optRight, middle - 1); ++split) {
                if (previous[split] == inf) continue;
                ll value = previous[split] + cost[split][middle];
                if (value < current[middle]) {
                    current[middle] = value;
                    best = split;
                }
            }
            self(self, left, middle - 1, optLeft, best);
            self(self, middle + 1, right, best, optRight);
        };
        solve(solve, layer, n, layer - 1, n - 1);
        previous.swap(current);
    }
    return previous;
}
```

65 Оптимизация Кнута: слияние соседних отрезков

По массиву неотрицательных весов возвращает минимальную стоимость их слияния в один блок: объединять можно только соседние блоки, цена равна сумме их весов. Пустой массив даёт 0; все суммы и стоимости должны помещаться в $[0, \text{LLONG_MAX}/4)$, отрицательные веса недопустимы. Время и память $O(n^2)$.

```
ll knuthMerge(const vector<ll>& weights) {
    int n = (int)weights.size();
    if (n == 0) return 0;
    vector<ll> prefix(n + 1);
    for (int i = 0; i < n; ++i) {
        assert(weights[i] >= 0);
        prefix[i + 1] = prefix[i] + weights[i];
    }
    vector<vector<ll>> dp(n, vector<ll>(n));
    vector<vector<int>> opt(n, vector<int>(n));
    for (int i = 0; i < n; ++i) opt[i][i] = i;
    for (int length = 2; length <= n; ++length) {
        for (int l = 0; l + length <= n; ++l) {
            int r = l + length - 1;
            dp[l][r] = LLONG_MAX / 4;
            for (int s = opt[l][r - 1]; s <= min(r - 1, opt[l + 1][r]); ++s) {
                ll value = dp[l][s] + dp[s + 1][r] + prefix[r + 1] - prefix[l];
                if (value < dp[l][r]) { dp[l][r] = value; opt[l][r] = s; }
            }
        }
    }
    return dp[0][n - 1];
}
```

66 Сканирующая прямая: площадь объединения прямоугольников

По прямоугольникам $\{x_1, y_1, x_2, y_2\}$ возвращает площадь их объединения, игнорируя вырожденные прямоугольники. Нужны $x_1 \le x_2$, $y_1 \le y_2$; координаты, все их разности, произведения ширины полосы на покрытую длину и итоговая площадь должны помещаться в ll. Время $O(n \log n)$, память $O(n)$.

```
ll rectangleUnionArea(const vector<array<ll, 4>>& rectangles) {
    struct Event { ll x, low, high; int delta; };
    vector<Event> events;
    vector<ll> ys;
```

```
    for (auto r : rectangles) {
        assert(r[0] <= r[2] && r[1] <= r[3]);
        if (r[0] == r[2] || r[1] == r[3]) continue;
        events.push_back({r[0], r[1], r[3], 1});
        events.push_back({r[2], r[1], r[3], -1});
        ys.push_back(r[1]); ys.push_back(r[3]);
    }
    if (events.empty()) return 0;
    sort(ys.begin(), ys.end()); ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(events.begin(), events.end(), [](const Event& a, const Event& b) { return a.x < b.x; });
    int segments = (int)ys.size() - 1;
    vector<int> cover(4 * segments);
    vector<ll> length(4 * segments);
    auto update = [&](auto&& self, int node, int l, int r, int ql, int qr, int delta) -> void {
        if (qr <= l || r <= ql) return;
        int middle = (l + r) / 2;
        if (ql <= l && r <= qr) cover[node] += delta;
        else { self(self, node * 2, l, middle, ql, qr, delta);
               self(self, node * 2 + 1, middle, r, ql, qr, delta); }
        if (cover[node] > 0) length[node] = ys[r] - ys[l];
        else length[node] = r - l == 1 ? 0 : length[node * 2] + length[node * 2 + 1];
    };
    ll area = 0, previous = events[0].x;
    int i = 0;
    while (i < (int)events.size()) {
        ll x = events[i].x;
        area += (x - previous) * length[1];
        while (i < (int)events.size() && events[i].x == x) {
            auto e = events[i++];
            int l = (int)(lower_bound(ys.begin(), ys.end(), e.low) - ys.begin());
            int r = (int)(lower_bound(ys.begin(), ys.end(), e.high) - ys.begin());
            update(update, 1, 0, segments, l, r, e.delta);
        }
        previous = x;
    }
    return area;
}
```

67 Двойной полиномиальный хеш подстрок

По строке и запросам $\{l, r, u, v\}$ сравнивает подстроки $[l, r)$ и $[u, v)$, возвращая логические ответы в порядке запросов. Индексы должны быть корректны, размеры — помещаться в `int`; пустые подстроки допустимы, предобработка занимает $O(n)$, запрос $O(1)$, память $O(n + q)$. Два фиксированных хеша могут совпасть у разных подстрок, особенно на специально подобранном входе; для точного подтверждения равенства сравните сами подстроки.

```
vector<bool> substring_equal(const string& text,
        const vector<array<int, 4>>& queries) {
    int n = int(text.size());
    const array<ll, 2> modulus{1000000007, 1000000009};
    const ll base = 911382323;
    vector<array<ll, 2>> prefix(n + 1), power(n + 1);
    power[0] = {1, 1};
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < 2; ++j) {
            power[i + 1][j] = power[i][j] * base % modulus[j];
            prefix[i + 1][j] = (prefix[i][j] * base
                + (unsigned char)text[i] + 1) % modulus[j];
        }
    auto hash = [&](int left, int right, int j) {
        return (prefix[right][j] - prefix[left][j]
            * power[right - left][j] % modulus[j] + modulus[j]) % modulus[j];
    };
    vector<bool> answer;
    for (auto [left, right, otherLeft, otherRight] : queries) {
        bool equal = right - left == otherRight - otherLeft;
        for (int j = 0; j < 2; ++j)
            equal = equal && hash(left, right, j) == hash(otherLeft, otherRight, j);
        answer.push_back(equal);
    }
    return answer;
}
```